

AI Employee following the hackathon document and the skills architecture.

Personal AI Employee Hackathon: Complete Implementation Guide Using Claude Code Skills

Executive Summary

This guide synthesizes the hackathon document with the Claude Code Skills architecture to help you build an autonomous Digital FTE (Full-Time Equivalent). The key innovation is implementing all AI functionality as reusable **Agent Skills** rather than monolithic scripts, making your system modular, maintainable, and extensible.

Current Date: 2026-01-28

Your Architecture Stack:

- **Brain:** Claude Code with Agent Skills (modular instructions)
 - **Memory:** Obsidian (local-first markdown vault)
 - **Sensors:** Python Watchers + Playwright for automation
 - **Hands:** MCP servers (Model Context Protocol)
 - **Orchestration:** Orchestrator.py + Watchdog.py
-

Part 1: Setting Up Your Skills Architecture

Understanding Agent Skills vs Traditional Prompts

Agent Skills are structured instructions that Claude loads dynamically. Unlike monolithic prompts, skills are **modular, reusable, and discoverable**. Each skill lives in its own directory with a `SKILL.md` file containing YAML frontmatter and instructions.

Structure for Personal AI Employee:

```
.claude/skills/
├── vault-operations/
│   ├── SKILL.md          # Core vault reading/writing
│   ├── templates/
│   │   ├── plan-template.md
│   │   ├── approval-template.md
│   │   └── briefing-template.md
│   └── examples/
│       ├── plan-example.md
│       └── briefing-example.md
├── gmail-processor/
│   ├── SKILL.md
│   ├── reference.md      # Gmail API details
│   └── examples/
│       └── email-processing-flow.md
├── whatsapp-handler/
│   ├── SKILL.md
│   └── examples/
│       └── message-response.md
├── financial-auditor/
│   ├── SKILL.md
│   ├── templates/
│   │   └── ceo-briefing.md
│   └── reference.md      # Accounting logic
├── approval-workflow/
│   ├── SKILL.md
│   └── examples/
│       └── approval-flow.md
└── ralph-wiggum-loop/
    ├── SKILL.md
    └── reference.md      # Task completion patterns
```

Part 2: Core Skills Implementation

Skill 1: Vault Operations (Foundation)

This skill teaches Claude how to interact with your Obsidian vault — the single source of truth for your AI Employee.

File: `.claude/skills/vault-operations/SKILL.md`

Copy

name: vault-operations

description: Read from and write to the Obsidian vault with proper folder structure and markdown formatting. Use when processing inbox items, creating plans, generating reports, or updating dashboards.

allowed-tools: Read, Write, Glob

Vault Operations Skill

Your personal Obsidian vault is the memory and control center for the AI Employee. Always interact with files using this consistent structure.

Vault Directory Structure

- `/Needs\_Action/` - Inbox of tasks from Watchers
- `/In\_Progress/<agent>/` - Tasks currently being worked on (claim by moving here)
- `/Plans/` - Generated plans with checklists (prefixed PLAN_*)
- `/Done/` - Completed tasks (moved after execution)
- `/Pending\_Approval/` - Actions requiring human approval (prefixed with action type)
- `/Approved/` - User-approved files ready for execution
- `/Rejected/` - User-rejected actions (log and skip)
- `/Logs/` - Audit trail in YYYY-MM-DD.json format
- `/Dashboard.md` - Real-time summary of system status
- `/Company\_Handbook.md` - Rules of engagement and thresholds
- `/Business\_Goals.md` - Quarterly objectives and metrics
- `/Accounting/Current\_Month.md` - Transaction ledger

Reading from Vault

When processing Needs_Action items:

1. **List all files**: Use Glob to scan `/Needs\_Action/\*.md`
2. **Parse metadata**: Extract YAML frontmatter (type, priority, source, created timestamp)
3. **Extract content**: Read the main content below the metadata
4. **Deduplicate**: Check `/In\_Progress/` to avoid duplicate processing
5. **Report**: List what you found with counts by type

Example: "Found 3 Gmail items, 1 WhatsApp message, 2 file drops"

Creating Action Files

When you need to take an action, **create an approval request first** if it touches external systems:

Approval File Pattern

```
``yaml
```

```
---
```

```
type: approval_request
action: <send_email|make_payment|post_social|create_invoice>
created: <ISO timestamp>
expires: <ISO timestamp + 24h>
status: pending
priority: <high|normal|low>
```

```
---
```

```
## Details
```

```
[Action-specific details]
```

```
## To Approve
```

```
Move this file to /Approved/
```

```
## To Reject
```

```
Move this file to /Rejected/
```

```
Copy
```

Plan File Pattern

```
Copy
```

```
---
```

```
type: plan
source: <WHATSAPP_client_a|EMAIL_vendor|etc>
created: <ISO timestamp>
status: pending_approval
```

```
---
```

```
## Objective
```

```
[What needs to be accomplished]
```

```
## Analysis
```

```
[Why this is needed, context from Company_Handbook]
```

```
## Steps
```

- [] Step 1 with specific details
- [] Step 2 with specific details
- [] Step 3 with specific details

```
## Approval Required
```

```
[What approval is needed before proceeding]
```

Completing Tasks

Once a task is approved and executed:

- **Update logs:** Add entry to /Logs/YYYY-MM-DD.json
- **Move files:** From /In_Progress/<you> → /Done/
- **Update Dashboard:** Add to "Recent Activity" section
- **Clean up:** Remove from /Needs_Action/

Log format:

Copy

```
{
  "timestamp": "2026-01-28T10:30:00Z",
  "task_id": "EMAIL_abc123",
  "action": "send_email",
  "status": "success",
  "details": {
    "to": "client@example.com",
    "subject": "Invoice #1234"
  },
  "approved_by": "human"
}
```

Dashboard Updates

The Dashboard.md is updated after each major action:

Copy

Real-Time Status

- **Latest update**: ISO timestamp
- **Bank balance**: \$X,XXX
- **Pending approvals**: N items
- **This week's revenue**: \$X,XXX

Recent Activity

- [Timestamp] Action description with outcome

Bottlenecks

- Task name: expected vs actual time

- Root cause or blocker

Error Handling

If a file is malformed or missing fields:

- **Quarantine:** Move to /Logs/ERRORS/ with timestamp suffix
- **Alert:** Log the error with context
- **Continue:** Process other items

See reference.md for detailed vault schema and examples for complete workflows.

```
---  
### **Skill 2: Gmail Processor**
```

```
**File: `.claude/skills/gmail-processor/SKILL.md`**
```

```
```yaml
```

```

name: gmail-processor
description: Process unread Gmail messages from the Watcher, categorize by urgency, draft
responses, and create approval requests for sending. Use when inbox items from Gmail Watcher
need intelligent triage.
allowed-tools: Read, Write, Grep

```

```
Gmail Processor Skill
```

This skill processes emails captured by the Gmail Watcher and determines appropriate responses.

```
Email File Format (from Watcher)
```

```
```yaml
```

```
---  
type: email  
from: sender@example.com  
subject: Email subject  
received: ISO timestamp  
message_id: unique_id  
priority: high|normal|low
```

status: pending

Email Content

[Full email body]

Metadata

- Is this from a known contact? (check Company_Handbook)
- Does it require urgent response? (check within Company_Handbook SLA)
- Is this a common category? (invoice request, question, etc)

Processing Steps

- **Categorize:**
 - Invoice/payment request
 - Client question/support
 - Administrative/notification
 - Spam/low priority
- **Check Company_Handbook Rules:**
 - Known clients get faster response
 - Payments over \$500 require approval
 - Response templates for common categories
- **Create Response Plan:**
 - Draft email in Plan.md
 - Flag for approval if needed
 - Include template if applicable
- **Move to Approval:**
 - From /Needs_Action/EMAIL_* →
/Plans/PLAN_email_response_*
 - Human reviews and moves to /Approved/

- When approved, Email MCP executes send

Draft Response Template

Always draft before sending to external contacts:

Copy

****To****: \$FROM

****Subject****: RE: \$SUBJECT

****Body****:

[Personalized response following Company_*Handbook tone*]

[*Signature per handbook*]

Special Cases

- **New contact**: Always create approval request
- **Bulk recipients**: Flag for review
- **Legal/sensitive**: Move to /Pending_Approval/ with
ALWAYS_REQUIRE_APPROVAL flag
- **Unread > 48h old**: Check if still relevant before responding

See reference.md for Gmail API details and examples/ for response patterns.

****Skill 3: Financial Auditor (CEO Briefing)****

****File: `.claude/skills/financial-auditor/SKILL.md`****

```yaml

name: financial-auditor

description: Generate Monday Morning CEO Briefing by analyzing revenue, tasks completed, and bottlenecks from the past week. Identify cost optimization opportunities. Use weekly as a scheduled task.

disable-model-invocation: false
allowed-tools: Read, Write, Grep, Glob

Financial Auditor Skill

This skill generates the "Monday Morning CEO Briefing" — a proactive analysis of your business health.

Weekly Audit Process

Trigger: Every Sunday 11pm → Monday 7am

Inputs:

1. ``/Business_Goals.md`` - Target metrics and objectives
2. ``/Accounting/Current_Month.md`` - Transaction ledger
3. ``/Tasks/Done/`` - All completed tasks this week
4. ``/Pending_Approval/`` - Stalled approvals

Process:

1. **Calculate Weekly Revenue**:
 - Sum invoices marked "paid" from Accounting
 - Filter by week boundaries
 - Compare to Business_Goals target
2. **Analyze Completed Tasks**:
 - Read all files in ``/Tasks/Done/`` with timestamp from past 7 days
 - Count by category (client work, admin, projects)
 - Calculate actual vs estimated time
3. **Identify Bottlenecks**:
 - Tasks taking > 2x expected time
 - Stalled approvals > 24h old
 - Recurring blockers from previous weeks
4. **Find Cost Savings**:
 - Parse Accounting for subscription patterns
 - Check usage logs: "last login 45+ days ago" = cancel candidate
 - Aggregate by vendor
 - Flag if monthly spend > 10% increase from previous month
5. **Generate Briefing**:
 - Write to ``/Briefings/2026-01-27_Monday_Briefing.md``
 - Include Executive Summary, Revenue, Tasks, Bottlenecks, Suggestions

CEO Briefing Template

See templates/ceo-briefing.md

Proactive Suggestions

After analysis, propose actions:

```markdown

#### Cost Optimization

- **Notion**: No activity 45 days. Cost: \$15/month → **ACTION**: Move to /Pending\_Approval/CANCEL\_NOTION.md
- **Adobe**: 20% price increase. Consider alternative? → **ACTION**: Investigate

#### Upcoming Deadlines

- **Project Alpha final**: 9 days
- **Tax prep**: 25 days

#### Team Insights

- **Fastest completed task**: Type A (avg 30 min)
- **Most delayed**: Type B (avg 3 days, target 1 day)
- **Recommendation**: Delegate Type B or break into smaller chunks

# Accounting Pattern Matching

Use this mapping to categorize transactions:

Copy

```
SUBSCRIPTION_PATTERNS = {
 'netflix.com': ('Entertainment', '$9.99'),
 'slack.com': ('Productivity', '$12.50'),
 'openai.com': ('AI Services', 'variable'),
 'stripe.com': ('Payment Processing', 'variable'),
 # ... add your common subscriptions
}
```

# Error Handling

- Missing Accounting data: Note gap and estimate from last available data
- No completed tasks: Flag as low-activity week
- Outdated Business\_Goals: Use last known targets with warning

See reference.md for full accounting taxonomy and examples/ for sample briefings.

---

#### \*\*Skill 4: Approval Workflow Manager\*\*

\*\*File: `.claude/skills/approval-workflow/SKILL.md`\*\*

``yaml

---

name: approval-workflow

description: Manage human-in-the-loop approval workflow. Monitor /Pending\_Approval/ folder, present items for user review, handle approvals and rejections, and update the audit log.

disable-model-invocation: false

allowed-tools: Read, Write, Glob

---

# Approval Workflow Manager Skill

Implements human-in-the-loop (HITL) safety checks for sensitive actions.

## Approval Categories & Thresholds

#### Auto-Approve (No HITL Required)

- Email to known contacts (in Company\_Handbook)
- Payments < \$50 (recurring subscriptions)
- Social media posts (scheduled, non-reply)
- File operations (create/read)

#### Always Require Approval

- Payments to new recipients
- Payments > \$100
- Email to new contacts
- Social media replies/DMs
- Delete operations
- External API calls with side effects

## Approval File Lifecycle

Created (auto) → /Pending\_Approval/ACTION\_\*.md ↓ Human reviews and makes decision ↓ /Approved/ OR /Rejected/ ↓ Orchestrator picks up and executes/logs

### ## Approval Request Format

Always include:

- **\*\*action\*\***: Type of action (send\_email, make\_payment, post\_social, etc)
- **\*\*amount\*\***: (for payments) Exact amount
- **\*\*recipient\*\***: (for payments/emails) Who receives this
- **\*\*reason\*\***: Why this action is needed
- **\*\*timestamp\*\***: When created
- **\*\*expires\*\***: When approval request expires (24h default)

### ## Handling Approvals

When user moves file to `/Approved/`:

1. **\*\*Validate\*\***: Check approval isn't expired
2. **\*\*Execute\*\***: Trigger appropriate MCP handler
3. **\*\*Log\*\***: Write success/failure to audit log
4. **\*\*Archive\*\***: Move to `/Done/` with timestamp suffix

Success log entry:

```
```json
{
  "timestamp": "2026-01-28T14:30:00Z",
  "action": "payment_sent",
  "recipient": "vendor@example.com",
  "amount": 250.00,
  "approval_status": "approved",
  "approved_by": "human",
  "result": "success"
}
```

Handling Rejections

When user moves file to `/Rejected/`:

- **Archive**: Move to `/Logs/REJECTED/` with timestamp
- **Log**: Record rejection
- **Notify**: Update Dashboard with rejection note
- **Stop**: Don't attempt this action again

Rejection log entry:

Copy

```
{
  "timestamp": "2026-01-28T14:35:00Z",
  "action": "payment_proposed",
  "recipient": "vendor@example.com",
  "reason": "Insufficient funds",
  "approval_status": "rejected",
  "result": "abandoned"
}
```

Email to Approval Example

When Gmail Processor drafts an email to **new contact**:

File: /Pending_Approval/EMAIL_new_vendor_inquiry.md

Copy

```
---
type: approval_request
action: send_email
to: vendor@example.com
subject: Inquiry about your services
created: 2026-01-28T10:30:00Z
expires: 2026-01-29T10:30:00Z
status: pending
priority: normal
---
```

Message Preview

To: vendor@example.com
Subject: Inquiry about your services

Body:
Hi there,

[Draft message]

Best regards,
Me

User Action

- To send: Move to /Approved/
- To reject: Move to /Rejected/

User moves to /Approved/EMAIL_ . . . → Orchestrator executes → logged to audit trail.

Payment to New Recipient Example

File: /Pending_Approval/PAYMENT_new_vendor_2026-01-28.md

Copy

```
---
type: approval_request
action: make_payment
amount: 250.00
recipient: vendor@example.com
reason: Invoice #5678 for software services
created: 2026-01-28T11:00:00Z
expires: 2026-01-29T11:00:00Z
status: pending
---
```

Payment Details

```
- **Amount**: $250.00
- **Recipient**: vendor@example.com (Bank: NEW)
- **Reference**: Invoice #5678
- **Due**: 2026-01-30
```

User Action

```
- To approve payment: Move to /Approved/
- To reject payment: Move to /Rejected/
```

See examples/ for more approval patterns.

```
### **Skill 5: Ralph Wiggum Loop (Task Completion)**
```

```
**File: `.claude/skills/ralph-wiggum-loop/SKILL.md`**
```

```
```yaml
```

```
name: ralph-wiggum-loop
```

**description:** Autonomous task completion loop. Keep working on a task until it's marked complete in the vault. Check /Done/ folder to verify completion, retry if needed, exit when task is complete or max iterations reached.

**disable-model-invocation:** true

**allowed-tools:** Read, Write, Glob

---

## # Ralph Wiggum Loop Skill

The "Ralph Wiggum Stop Hook" is a pattern that keeps Claude iterating until a task is complete. Rather than exiting after one attempt, it loops until the task file moves to `/Done/`.

### ## How It Works

1. **\*\*Task Created\*\***: Orchestrator.py writes state file to `/Tasks/TASK\_<id>.md`
2. **\*\*Claude Works\*\***: You read task, execute steps, write progress to vault
3. **\*\*Check Completion\*\***: At end, check if task file moved to `/Done/`
4. **\*\*Loop or Exit\*\***:
  - If in `/Done/`: Task complete, exit
  - If still in `/In\_Progress/`: Check max iterations, continue if < 10
  - If stuck: Log error and abort

### ## Task State File Format

```yaml

type: task

task_id: process_weekly_inbox

created: 2026-01-28T08:00:00Z

status: in_progress

max_iterations: 10

current_iteration: 1

Objective

Process all items in /Needs_Action/ and move completed items to /Done/

Steps

- [] List all files in /Needs_Action/
- [] Process each file by type
- [] Create plans/approvals as needed
- [] Move processed files to /Done/
- [] Update Dashboard.md
- [] **COMPLETION**: All /Needs_Action items processed

Iteration Pattern

Iteration 1:

- List files in `/Needs_Action/`
- Find 5 items to process
- Start processing first item

Iteration 2:

- Check progress file
- Continue with next items
- Create approval files

Iteration 3:

- All items processed
- Move task to `/Done/TASK_process_weekly_inbox_2026-01-28.md`
- Exit (completion detected)

Loop Detection

Prevent infinite loops:

Copy

```
# In orchestrator
MAX_ITERATIONS = 10
if current_iteration > MAX_ITERATIONS:
    move_to_done(task_id, "MAX_ITERATIONS_EXCEEDED")
    exit_with_error()
```

Completion Strategies

Strategy 1: File Movement (Recommended)

Task is complete when file moves to /Done/:

Copy

```
# Orchestrator checks
if [ -f /Vault/Done/TASK_${TASK_ID}.md ]; then
    echo "Task complete"
    exit 0
else
    echo "Task in progress, retrying..."
    re_inject_prompt
fi
```

Strategy 2: Marker in File

Task includes completion marker:

Copy

```
# In task file, you write:
---
status: complete
completion_marker: INVOICE_BATCH_COMPLETE_V2
---
```

Orchestrator greps for marker and exits.

Usage Example

Copy

```
# Orchestrator.py calls this
claude --cwd /path/to/vault \
    --skill ralph-wiggum-loop \
    "Process all /Needs_Action items until /Done/"

# Claude runs and iterates until task moves to /Done/
```

Error Cases

If task never moves to /Done/

- Max iterations hit
- Log final state

- Move to /Logs/INCOMPLETE/
- Alert human

If task is in error state

- Quarantine related files
- Log error with context
- Require manual intervention

See reference.md for implementation details.

Part 3: Implementing Watchers with Skill Integration

Base Watcher Template (Integrates with Skills)

File: `watchers/base\_watcher.py`

```

python
import time
import logging
from pathlib import Path
from abc import ABC, abstractmethod
import json
from datetime import datetime

class BaseWatcher(ABC):
    """
    Base class for all Watchers. A Watcher is a lightweight daemon that monitors
    external sources (Gmail, WhatsApp, bank accounts) and writes .md files to
    /Needs_Action/ for Claude to process.

    These files trigger Claude Code with the appropriate skills:
    - EMAIL_* files → gmail-processor skill
    - WHATSAPP_* files → whatsapp-handler skill
    - TRANSACTION_* files → financial-auditor skill
    """

    def __init__(self, vault_path: str, check_interval: int = 60):
        self.vault_path = Path(vault_path)

```

```

self.needs_action = self.vault_path / 'Needs_Action'
self.check_interval = check_interval
self.logger = logging.getLogger(self.__class__.__name__)
self.processed_items = set() # Track what we've already reported

# Ensure directories exist
self.needs_action.mkdir(parents=True, exist_ok=True)

@abstractmethod
def check_for_updates(self) -> list:
    """Return list of new items to process from external source"""
    pass

@abstractmethod
def create_action_file(self, item) -> Path:
    """Create .md file in Needs_Action folder"""
    pass

def run(self):
    """Main loop - run continuously"""
    self.logger.info(f'Starting {self.__class__.__name__}')
    while True:
        try:
            items = self.check_for_updates()
            for item in items:
                self.create_action_file(item)
                self.logger.info(f'Created action file for: {item}')
        except Exception as e:
            self.logger.error(f'Error in check loop: {e}', exc_info=True)

        time.sleep(self.check_interval)

def _create_markdown_file(self, filename: str, frontmatter: dict, content: str) -> Path:
    """Helper to create properly formatted .md file with YAML frontmatter"""
    # Build YAML frontmatter
    yaml_lines = ['---']
    for key, value in frontmatter.items():
        if isinstance(value, str):
            yaml_lines.append(f'{key}: {value}')
        else:
            yaml_lines.append(f'{key}: {json.dumps(value)}')
    yaml_lines.append('---')
    yaml_section = '\n'.join(yaml_lines)

    # Combine with content
    full_content = f'{yaml_section}\n\n{content}'

```

```
filepath = self.needs_action / filename
filepath.write_text(full_content)
return filepath
```

Gmail Watcher (Triggers Gmail Processor Skill)

File: `watchers/gmail_watcher.py`

Copy

```
from base_watcher import BaseWatcher
from google.oauth2.credentials import Credentials
from googleapiclient.discovery import build
from datetime import datetime
import os

class GmailWatcher(BaseWatcher):
    """
    Monitors Gmail for unread important messages.
    Creates EMAIL_* files that trigger the gmail-processor skill.
    """

    def __init__(self, vault_path: str, credentials_file: str, check_interval: int = 120):
        super().__init__(vault_path, check_interval)
        self.creds = Credentials.from_authorized_user_file(credentials_file)
        self.service = build('gmail', 'v1', credentials=self.creds)
        self.processed_ids = set()

    def check_for_updates(self) -> list:
        """Check for unread important emails"""
        results = self.service.users().messages().list(
            userId='me',
            q='is:unread is:important',
            maxResults=10
        ).execute()

        messages = results.get('messages', [])
        new_messages = [m for m in messages if m['id'] not in self.processed_ids]

        return new_messages

    def create_action_file(self, message) -> Path:
        """Create EMAIL_*.md file with email details"""
        msg_data = self.service.users().messages().get(
            userId='me',
```

```

        id=message['id'],
        format='full'
    ).execute()

    headers = {h['name']: h['value'] for h in msg_data['payload']['headers']}

    # Determine priority based on keywords
    subject = headers.get('Subject', 'No Subject').lower()
    snippet = msg_data.get('snippet', "").lower()
    text = f"{subject} {snippet}"

    priority = 'high' if any(kw in text for kw in ['urgent', 'asap', 'invoice']) else 'normal'

    frontmatter = {
        'type': 'email',
        'from': headers.get('From', 'Unknown'),
        'subject': headers.get('Subject', 'No Subject'),
        'received': datetime.now().isoformat(),
        'message_id': message['id'],
        'priority': priority,
        'status': 'pending'
    }

    content = f"""## Email Content

{msg_data.get('snippet', 'No preview available')}

## Suggested Actions
- [ ] Analyze and categorize
- [ ] Draft response (if needed)
- [ ] Create approval request
- [ ] Archive after processing
"""

    filename = f"EMAIL_{message['id']}.md"
    filepath = self._create_markdown_file(filename, frontmatter, content)
    self.processed_ids.add(message['id'])

    return filepath

if __name__ == '__main__':
    import logging
    logging.basicConfig(level=logging.INFO)

    vault_path = os.getenv('VAULT_PATH', '/path/to/AI_Employee_Vault')
    creds_file = os.getenv('GMAIL_CREDS', '/path/to/credentials.json')

```

```
watcher = GmailWatcher(vault_path, creds_file)
watcher.run()
```

Copy

WhatsApp Watcher (Using Playwright)

File: `watchers/whatsapp_watcher.py`

Copy

```
from base_watcher import BaseWatcher
from playwright.sync_api import sync_playwright
from pathlib import Path
from datetime import datetime
import os

class WhatsAppWatcher(BaseWatcher):
    """
    Monitors WhatsApp Web for urgent messages (keywords: invoice, payment, urgent, etc).
    Uses Playwright for browser automation.
    Creates WHATSAPP_* files that trigger whatsapp-handler skill.
    """

    def __init__(self, vault_path: str, session_path: str, check_interval: int = 30):
        super().__init__(vault_path, check_interval)
        self.session_path = Path(session_path)
        self.keywords = ['invoice', 'payment', 'urgent', 'asap', 'help', 'order', 'quote']
        self.last_seen_ids = set()

    def check_for_updates(self) -> list:
        """Check for unread WhatsApp messages containing keywords"""
        messages = []

        try:
            with sync_playwright() as p:
                browser = p.chromium.launch_persistent_context(
                    str(self.session_path),
                    headless=True
                )

            if not browser.pages:
                self.logger.warning("No pages in persistent context")
                return messages

            page = browser.pages[0]
```

```

page.goto('https://web.whatsapp.com', timeout=30000)

try:
    page.wait_for_selector('[data-testid="chat-list"]', timeout=10000)
except:
    self.logger.warning("Could not find chat list")
    browser.close()
    return messages

# Find unread chats
unread_chats = page.query_selector_all('[aria-label*="unread"]')

for chat in unread_chats:
    try:
        text = chat.inner_text().lower()
        if any(kw in text for kw in self.keywords):
            # Extract sender and message preview
            sender_elem = chat.query_selector('[data-testid="cell-frame-title"]')
            sender = sender_elem.inner_text() if sender_elem else "Unknown"

            msg_id = f"{sender}_{datetime.now().timestamp()}"
            if msg_id not in self.last_seen_ids:
                messages.append({
                    'sender': sender,
                    'preview': text[:200],
                    'id': msg_id,
                    'timestamp': datetime.now().isoformat()
                })
                self.last_seen_ids.add(msg_id)
            except Exception as e:
                self.logger.warning(f"Error processing chat: {e}")

    browser.close()

except Exception as e:
    self.logger.error(f"Error in WhatsApp watcher: {e}", exc_info=True)

return messages

def create_action_file(self, message) -> Path:
    """Create WHATSAPP_*.md file"""
    frontmatter = {
        'type': 'whatsapp_message',
        'from': message['sender'],
        'received': message['timestamp'],
        'message_id': message['id'],

```

```

        'priority': 'high',
        'status': 'pending'
    }

    content = f"""## Message Preview

{message['preview']}

## Suggested Actions
- [ ] Read full message
- [ ] Determine response type
- [ ] Draft reply
- [ ] Send (with approval if needed)
"""

    filename = f"WHATSAPP_{message['id']}.md"
    return self._create_markdown_file(filename, frontmatter, content)

if __name__ == '__main__':
    import logging
    logging.basicConfig(level=logging.INFO)

    vault_path = os.getenv('VAULT_PATH', '/path/to/AI_Employee_Vault')
    session_path = os.getenv('WHATSAPP_SESSION', '/path/to/whatsapp_session')

    watcher = WhatsAppWatcher(vault_path, session_path)
    watcher.run()

```

Copy

Part 4: Orchestrator Architecture

File: orchestrator.py — Master coordination engine

Copy

```

#!/usr/bin/env python3
"""

```

Orchestrator.py - Master coordination engine for Personal AI Employee.

Responsibilities:

1. Schedule Claude to run with appropriate skills
2. Monitor vault for task updates
3. Watch for approvals and execute MCPs
4. Manage process lifecycle

5. Log all actions

"""

```
import subprocess
import time
import json
from pathlib import Path
from datetime import datetime, timedelta
import schedule
import logging
from typing import Optional
import os

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('/path/to/Al_Employee_Vault/Logs/orchestrator.log'),
        logging.StreamHandler()
    ]
)
logger = logging.getLogger('Orchestrator')
```

```
class PersonalAIOrchestrator:
    def __init__(self, vault_path: str):
        self.vault_path = Path(vault_path)
        self.needs_action = self.vault_path / 'Needs_Action'
        self.approved = self.vault_path / 'Approved'
        self.done = self.vault_path / 'Done'
        self.logs = self.vault_path / 'Logs'

        # Ensure directories exist
        for d in [self.needs_action, self.approved, self.done, self.logs]:
            d.mkdir(parents=True, exist_ok=True)

    def schedule_tasks(self):
        """Set up daily/weekly scheduled tasks"""

        # Daily briefing at 8 AM
        schedule.every().monday.at("08:00").do(
            self.run_skill_with_task,
            skill='financial-auditor',
            task_description='Generate Monday Morning CEO Briefing'
        )

        # Process inbox every 2 hours
```

```

schedule.every(2).hours.do(
    self.run_skill_with_task,
    skill='vault-operations',
    task_description='Process /Needs_Action inbox'
)

# Approval check every 30 minutes
schedule.every(30).minutes.do(self.check_approvals)

def run_skill_with_task(self, skill: str, task_description: str):
    """
    Run Claude with a specific skill using Ralph Wiggum loop pattern.

    Args:
        skill: Skill name (e.g., 'financial-auditor', 'gmail-processor')
        task_description: Description of task for Claude
    """
    task_id = f"{skill}_{datetime.now().strftime('%Y%m%d_%H%M%S')}"
    task_file = self.vault_path / f"TASK_{task_id}.md"

    # Create task state file
    task_state = f"""---
type: task
task_id: {task_id}
created: {datetime.now().isoformat()}
skill: {skill}
status: in_progress
max_iterations: 10
current_iteration: 0
---

## Objective
{task_description}

## Status
Created at {datetime.now().isoformat()}

## Progress
(Claude will update this section)
"""

    task_file.write_text(task_state)
    logger.info(f"Created task: {task_id}")

    # Call Claude with skill
    try:
        cmd = [

```

```

        'claude',
        '--cwd', str(self.vault_path),
        '--skill', skill,
        f'Process this task: {task_description}'
    ]

    result = subprocess.run(cmd, capture_output=True, text=True, timeout=600)

    if result.returncode == 0:
        logger.info(f"Skill {skill} completed successfully")
        # Move task to done
        done_file = self.done / f"TASK_{task_id}.md"
        if task_file.exists():
            task_file.rename(done_file)
        self.log_action('skill_executed', skill, 'success')
    else:
        logger.error(f"Skill {skill} failed: {result.stderr}")
        self.log_action('skill_executed', skill, 'failure', result.stderr)

    except subprocess.TimeoutExpired:
        logger.error(f"Skill {skill} timed out")
        self.log_action('skill_executed', skill, 'timeout')
    except Exception as e:
        logger.error(f"Error running skill {skill}: {e}")
        self.log_action('skill_executed', skill, 'error', str(e))

def check_approvals(self):
    """Check for approved actions and execute via MCP"""
    if not self.approved.exists():
        return

    approved_files = list(self.approved.glob("*.md"))
    for f in approved_files:
        try:
            content = f.read_text()
            # Parse frontmatter to get action type
            if content.startswith('---'):
                yaml_end = content.find('---', 3)
                yaml_section = content[4:yaml_end]

                action_type = None
                if 'action: send_email' in yaml_section:
                    action_type = 'send_email'
                elif 'action: make_payment' in yaml_section:
                    action_type = 'make_payment'
                elif 'action: post_social' in yaml_section:

```

```

        action_type = 'post_social'

    if action_type:
        self.execute_mcp(f, action_type)
        # Move to Done after execution
        f.rename(self.done / f.name)
        logger.info(f"Executed and archived: {f.name}")

    except Exception as e:
        logger.error(f"Error processing approval {f.name}: {e}")

def execute_mcp(self, approval_file: Path, action_type: str):
    """Execute action via appropriate MCP server"""
    # This integrates with your MCP servers
    logger.info(f"Would execute MCP action: {action_type} for {approval_file.name}")
    # Implementation depends on your MCP setup

def log_action(self, action_type: str, detail: str, status: str, error: str = None):
    """Log all actions to audit trail"""
    log_file = self.logs / f"{datetime.now().strftime('%Y-%m-%d')}.json"

    entry = {
        'timestamp': datetime.now().isoformat(),
        'action_type': action_type,
        'detail': detail,
        'status': status,
        'error': error
    }

    # Append to daily log
    entries = []
    if log_file.exists():
        entries = json.loads(log_file.read_text())

    entries.append(entry)
    log_file.write_text(json.dumps(entries, indent=2))

def start(self):
    """Main orchestrator loop"""
    self.schedule_tasks()
    logger.info("Personal AI Employee Orchestrator starting")

    while True:
        schedule.run_pending()
        time.sleep(60)

```

```
if __name__ == '__main__':  
    vault_path = os.getenv('VAULT_PATH', '/path/to/AI_Employee_Vault')  
    orchestrator = PersonalAIOrchestrator(vault_path)  
    orchestrator.start()
```

Copy

Part 5: Project Structure & Quick Start

```
ai_employee_project/  
├── .claude/  
│   └── skills/  
│       ├── vault-operations/  
│       │   ├── SKILL.md  
│       │   ├── examples/  
│       │   └── templates/  
│       ├── gmail-processor/  
│       │   ├── SKILL.md  
│       │   ├── reference.md  
│       │   └── examples/  
│       ├── whatsapp-handler/  
│       │   ├── SKILL.md  
│       │   └── examples/  
│       ├── financial-auditor/  
│       │   ├── SKILL.md  
│       │   ├── templates/  
│       │   └── reference.md  
│       ├── approval-workflow/  
│       │   ├── SKILL.md  
│       │   └── examples/  
│       └── ralph-wiggum-loop/  
│           ├── SKILL.md  
│           └── reference.md  
├── watchers/  
│   ├── base_watcher.py  
│   ├── gmail_watcher.py  
│   ├── whatsapp_watcher.py  
│   └── filesystem_watcher.py  
├── orchestrator.py  
├── watchdog.py  
├── AI_Employee_Vault/  
│   ├── Needs_Action/  
│   ├── Plans/  
│   ├── Done/  
│   └── Pending_Approval/
```

```
| — Approved/
| — In_Progress/
| — Logs/
| — Accounting/
| — Dashboard.md
| — Company_Handbook.md
| — Business_Goals.md
| — .env (add to .gitignore!)
| — requirements.txt
| — README.md
```

Part 6: Tier-Based Delivery Roadmap

Bronze Tier (8-12 hours)

- ✓ Create `.claude/skills/vault-operations/SKILL.md`
- ✓ Implement `watchers/base_watcher.py` + Gmail Watcher
- ✓ Create Obsidian vault with `/Needs_Action`, `/Plans`, `/Done` folders
- ✓ Write basic orchestrator that calls Claude with skills
- ✓ Implement `Company_Handbook.md` with rules

Demo: Create an email, have Claude process it and move to `/Done`

Silver Tier (20-30 hours)

- ✓ All Bronze requirements
- ✓ Implement all 5 core skills (vault-ops, gmail, whatsapp, financial-auditor, approval-workflow)
- ✓ Add WhatsApp Watcher with Playwright
- ✓ Implement approval workflow with human-in-the-loop
- ✓ Add Email MCP for sending
- ✓ Implement scheduling (2-hour inbox checks)

Demo: Email arrives → Gmail Processor skill → creates Plan → user approves → sent via MCP → logged

Gold Tier (40+ hours)

- ✓ All Silver requirements
- ✓ Implement Ralph Wiggum loop for multi-step autonomy
- ✓ Add Financial Auditor skill with full CEO Briefing
- ✓ Integrate Odoo Community for accounting
- ✓ Add Facebook/Instagram/Twitter skills
- ✓ Implement comprehensive audit logging
- ✓ Add error recovery and graceful degradation

Demo: Sunday night → Monday 7am CEO Briefing generated with revenue, bottlenecks, cost savings

Platinum Tier (60+ hours)

- ✓ All Gold requirements
- ✓ Deploy Cloud VM (Oracle/AWS free tier)
- ✓ Cloud agent handles drafting (email, social, accounting drafts)
- ✓ Local agent handles approvals and sends
- ✓ Implement vault sync (Git-based)
- ✓ Secrets never leave local machine
- ✓ Always-on orchestration with health monitoring

Demo: Email arrives while local is offline → Cloud drafts → user approves locally → Local executes → verified end-to-end

Part 7: Security Checklist

- Never commit `.env` file (add to `.gitignore`)
 - Use environment variables for all credentials
 - Implement dry-run mode for testing
 - Create separate Gmail/banking test accounts
 - Rate-limit actions (max 10 emails/hour, max 3 payments/hour)
 - Implement audit logging with 90-day retention
 - Use HITL for all payments over \$100
 - Rotate credentials monthly
 - Encrypt vault at rest (Obsidian supports this)
 - Review approval logs weekly
-

Part 8: Next Steps

- **Install prerequisites:** Python 3.13+, Node.js 24+ LTS, Obsidian v1.10.6+, Claude Code
- **Create vault structure:** `mkdir -p AI_Employee_Vault/{Needs_Action,Plans,Done,Logs,Accounting}`
- **Start with vault-operations skill:** Get comfortable reading/writing files with Claude
- **Add Gmail Watcher:** Test email capture and action file creation
- **Implement approval workflow:** Experience the HITL safety pattern
- **Join Wednesday Research Meetings:**
<https://us06web.zoom.us/j/87188707642?pwd=a9XloCsinvn1JzICbPc2YGUvWTbOTr.1>

This architecture transforms you from using prompts to **building AI systems**. Skills are modular, reusable, and discoverable — exactly what production AI needs. Your Personal AI Employee becomes not just a tool, but a genuine business partner.

can you convert this chat session into document

Personal AI Employee Hackathon: Complete Implementation Guide Using Claude Code Skills

A Comprehensive Document for Building Autonomous Digital FTEs in 2026

Document Date: January 28, 2026

Status: Complete Implementation Guide

Target Audience: Hackathon Participants (Bronze through Platinum Tiers)

Table of Contents

- Introduction & Overview
- Understanding Agent Skills Architecture
- Core Skills Implementation
- Watcher System Design
- Orchestrator Architecture
- Project Structure & Setup

- Tier-Based Delivery Roadmap
 - Security & Best Practices
 - Quick Start Guide
 - Troubleshooting & FAQ
-

Introduction & Overview

What is a Digital FTE?

A Digital Full-Time Equivalent (FTE) is an AI agent built, hired, and priced as if it were a human employee. This represents a fundamental shift in how we think about software — not as applications or features, but as **autonomous team members**.

Key Metrics:

| Aspect | Human FTE | Digital FTE |
|--------------|------------------|------------------------|
| Availability | 40 hours/week | 168 hours/week (24/7) |
| Monthly Cost | \$4,000–\$8,000+ | \$500–\$2,000 |
| Setup Time | 3–6 months | Instant (via SKILL.md) |

| | | |
|----------------------|--------------------------|-----------------------------------|
| Consistency | 85–95% accuracy | 99%+ consistency |
| Scaling | Linear (hire 10 for 10x) | Exponential (duplicate instantly) |
| Cost per Task | ~\$3.00–\$6.00 | ~\$0.25–\$0.50 |
| Annual Hours | ~2,000 hours | ~8,760 hours |

The Aha Moment: A Digital FTE works nearly 9,000 hours a year vs. a human's 2,000.

The cost-per-task reduction (from ~\$5 to ~\$0.50) represents an **85–90% cost saving** — typically the threshold where a CEO approves a project without further debate.

This Guide's Approach

Rather than building monolithic scripts, we implement all AI functionality as **reusable Agent Skills**. This architectural shift provides:

- **Modularity:** Each skill is self-contained and independently testable
- **Discoverability:** Claude automatically loads skills when relevant
- **Reusability:** Skills work across projects, contexts, and organizations
- **Maintainability:** Update one skill file instead of refactoring entire systems

Understanding Agent Skills Architecture

What are Agent Skills?

Agent Skills are modular packages of instructions, templates, and reference materials that teach Claude how to complete specific tasks repeatably. Instead of embedding all instructions in a conversation, skills live in dedicated files that Claude loads dynamically.

Traditional Approach (Monolithic):

Claude reads one **large** prompt

- Follows instructions
- Completes task
- **No** reuse, **no** discovery

Skills Approach (Modular):

Claude discovers available skills

- Loads relevant skill (SKILL.md)
- Reads templates, examples, **references as** needed
- Completes task
- Skill reusable across contexts

Skills Specification

Agent Skills follow an **open standard** that works across multiple AI platforms (Claude, GitHub Copilot, and others). Each skill is a directory containing:

```
my-skill/  
├── SKILL.md      # Required: main instructions + frontmatter  
├── reference.md  # Optional: detailed API/domain knowledge  
├── examples/  
│   └── example-1.md # Optional: concrete usage examples  
├── templates/  
│   └── template.md # Optional: Claude fills in these templates  
└── scripts/  
    └── helper.py  # Optional: executable utilities
```

The SKILL .md file is the entry point. It contains **YAML frontmatter** (configuration) followed by **markdown instructions** (what Claude should do).

Why Skills Matter for Your AI Employee

Your Personal AI Employee needs to juggle multiple domains simultaneously:

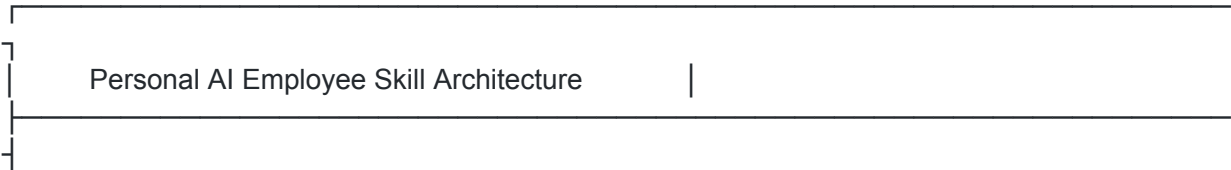
- Email processing (Gmail)
- Chat handling (WhatsApp)
- Financial analysis (banking)
- Accounting (Odoo integration)
- Social media posting (LinkedIn, Twitter, Facebook)
- Document generation (invoices, reports)
- Approval workflows (human safety)

Without skills, these responsibilities would create a bloated, hard-to-maintain prompt.
With skills, each domain is a self-contained module that Claude loads on-demand.

Core Skills Implementation

Architecture Overview: The Five Foundational Skills

Your Personal AI Employee runs on five core skills, each responsible for a specific functional domain:



Vault Operations (Memory/State Management)

- Read Needs_Action
- Create Plans and Approvals
- Move files between folders
- Update Dashboard

| | | |
|------------------|------------------|-----------------------|
| Gmail Processor | WhatsApp Handler | Financial Auditor |
| Categorize | Detect | Generate CEO Briefing |
| Create Plan | urgent msgs | |
| Draft reply | Create Plan | |
| Request approval | Request approval | Find cost savings |

Approval Workflow (Human-in-the-Loop Safety)

- Monitor /Pending_Approval/
- Handle user approvals/rejections
- Trigger MCP execution
- Audit log everything

Ralph Wiggum Loop (Task Completion)

- Keep working until task moves to /Done/
- Iterate up to 10 times
- Detect completion automatically

Skill 1: Vault Operations (Foundation)

Purpose: Teach Claude how to interact with your Obsidian vault — the single source of truth for your AI Employee.

File Location: `.claude/skills/vault-operations/SKILL.md`

Copy

```
---
name: vault-operations
description: Read from and write to the Obsidian vault with proper folder structure and markdown formatting. Use when processing inbox items, creating plans, generating reports, or updating dashboards.
allowed-tools: Read, Write, Glob
---
```

Vault Operations Skill

Your personal Obsidian vault is the memory and control center for the AI Employee. Always interact with files using this consistent structure.

Vault Directory Structure

The vault follows a clear workflow structure:

- `**/Needs_Action/**` — Inbox of tasks from Watchers (fresh items)
- `**/In_Progress/<agent>/**` — Tasks currently being worked on (claim by moving here)
- `**/Plans/**` — Generated plans with checklists (prefixed `PLAN_*`)
- `**/Done/**` — Completed tasks (moved after execution)
- `**/Pending_Approval/**` — Actions requiring human approval (prefixed with action type)
- `**/Approved/**` — User-approved files ready for execution
- `**/Rejected/**` — User-rejected actions (log and skip)
- `**/Logs/**` — Audit trail in `YYYY-MM-DD.json` format
- `**/Dashboard.md**` — Real-time summary of system status
- `**/Company_Handbook.md**` — Rules of engagement and thresholds
- `**/Business_Goals.md**` — Quarterly objectives and metrics
- `**/Accounting/Current_Month.md**` — Transaction ledger

Reading from Vault

When processing `Needs_Action` items:

Step 1: List All Files

Use `Glob` to scan `*/Needs_Action/*.md` and report what you found:

Copy

Found 3 Gmail items:

- EMAIL_abc123.md (High priority, from client@example.com)
- EMAIL_def456.md (Normal, invoice request)
- EMAIL_ghi789.md (Normal, newsletter)

Found 1 WhatsApp message:

- WHATSAPP_msg_001.md (High priority, "need quote ASAP")

Found 2 file drops:

- FILE_report.pdf
- FILE_invoice.xlsx

Step 2: Parse Metadata

Extract YAML frontmatter from each file. Every action file includes:

```
```yaml

type: email | whatsapp_message | file_drop
from: <sender>
subject: <subject or description>
received: <ISO timestamp>
priority: high | normal | low
status: pending

```

**Step 3: Extract Content**

Read the main content below the metadata separator (---).

#### Step 4: Deduplicate

Check `/In_Progress/` to avoid duplicate processing.

#### Step 5: Report

Always report what you found before taking action.

## Creating Action Files

When you need to take an action, **create an approval request first** if it touches external systems.

#### Approval File Pattern

Every approval request follows this format:

Copy

---

```
type: approval_request
action: send_email | make_payment | post_social | create_invoice
created: <ISO timestamp>
expires: <ISO timestamp + 24h>
status: pending
priority: high | normal | low
```

---

## Details

[Action-specific details here]

## To Approve

Move this file to `/Approved/`

## To Reject

Move this file to `/Rejected/`

## Plan File Pattern

Plans document your analysis and next steps:

Copy

---

type: plan

source: <original\_action\_id>

created: <ISO timestamp>

status: pending\_approval

---

## Objective

[What needs to be accomplished]

## Analysis

[Why this is needed, context from Company\_Handbook]

## Steps

- [ ] Step 1 with specific details

- [ ] Step 2 with specific details

- [ ] Step 3 with specific details

## Approval Required

[What approval is needed before proceeding]

## Completing Tasks

Once a task is approved and executed:

## 1. Update Logs

Add entry to /Logs/YYYY-MM-DD.json:

Copy

```
{
 "timestamp": "2026-01-28T10:30:00Z",
 "task_id": "EMAIL_abc123",
 "action": "send_email",
 "status": "success",
 "details": {
 "to": "client@example.com",
 "subject": "Invoice #1234"
 },
 "approved_by": "human"
}
```

## 2. Move Files

From /In\_Progress/<agent> → /Done/ with timestamp suffix

## 3. Update Dashboard

Add to "Recent Activity" section:

Copy

### ## Recent Activity

- [2026-01-28 10:30] Email sent to client@example.com (\$1,500 invoice)
- [2026-01-28 09:15] WhatsApp reply sent to Client A

## 4. Clean Up

Remove from /Needs\_Action/

# Dashboard Updates

The Dashboard.md is the real-time status page. Always update after major actions:

Copy

## ## Real-Time Status

- **Latest update**: 2026-01-28T14:30:00Z
- **Bank balance**: \$5,250
- **Pending approvals**: 2 items
- **This week's revenue**: \$2,450

## ## Recent Activity

- [2026-01-28 10:30] Invoice sent to Client A (\$1,500)
- [2026-01-28 09:15] WhatsApp reply sent to Client B
- [2026-01-27 16:00] Payment received from Client C (\$800)

## ## Bottlenecks

- Project Alpha delivery: Expected 2 days, took 5 days
  - Reason: Waiting for client feedback

# Error Handling

If a file is malformed or missing fields:

- **Quarantine**: Move to /Logs/ERRORS/ with timestamp suffix
- **Alert**: Log the error with context in the audit log
- **Continue**: Process other items

See reference.md for detailed vault schema and examples/ for complete workflows.

---

### Skill 2: Gmail Processor

**Purpose**: Process unread Gmail messages from the Watcher, categorize by urgency, draft responses, and create approval requests.

**\*\*File Location\*\*:** ``.claude/skills/gmail-processor/SKILL.md``

````yaml`

`---`

`name: gmail-processor`

`description: Process unread Gmail messages from the Watcher, categorize by urgency, draft responses, and create approval requests for sending. Use when inbox items from Gmail Watcher need intelligent triage.`

`allowed-tools: Read, Write, Grep`

`---`

`# Gmail Processor Skill`

This skill processes emails captured by the Gmail Watcher and determines appropriate responses.

`## Email File Format (from Watcher)`

The Gmail Watcher creates files like this in ``/Needs_Action/``:

````yaml`

`---`

`type: email`

`from: sender@example.com`

`subject: Email subject line`

`received: 2026-01-28T10:15:00Z`

`message_id: msg_abc123`

`priority: high | normal | low`

`status: pending`

`---`

`## Email Content`

[Full email body or snippet]

`## Metadata`

- Is this from a known contact? (check `Company_Handbook` `known_clients` list)
- Does it require urgent response? (check `Company_Handbook` SLA)
- Is this a common category? (invoice request, support question, etc)

## Processing Steps

## Step 1: Categorize the Email

Determine the email type:

- **Invoice/Payment Request:** Client asking for invoice or reporting payment
- **Client Question/Support:** Substantive question requiring thought
- **Administrative/Notification:** System alerts, receipts, newsletters
- **Spam/Low Priority:** Unsolicited marketing, old threads, etc.

Example categorization:

**From:** client@example.com

**Subject:** "Can you send me the invoice for January?"

**Category:** Invoice Request

**Priority:** High (revenue-related)

**Action:** Generate invoice **and** send

## Step 2: Check Company\_Handbook Rules

Look up policies in /Company\_Handbook.md:

- **Response SLA:** How quickly to respond?
- **Known Contacts:** Should be fast-tracked
- **Payment Thresholds:** Amounts requiring approval
- **Response Templates:** Standard responses for common scenarios
- **Tone Guidelines:** How to communicate

Example from handbook:

Copy

### ## Known Clients

- Client A (24h response SLA)
- Client B (24h response SLA)

## ## Payment Approvals

- Payments < \$50: Auto-approve
- Payments \$50-\$500: Require approval if new recipient
- Payments > \$500: Always require approval

## ## Response Templates

- Invoice Request: "See attached..."
- Quote Request: "Happy to discuss..."

## Step 3: Create Response Plan

Draft the response **before requesting approval**:

File: /Plans/PLAN\_email\_response\_<id>.md

Copy

---

type: plan  
source: EMAIL\_msg\_abc123  
created: 2026-01-28T10:30:00Z  
status: pending\_approval  
---

### ## Objective

Send invoice to Client A in response to their request.

### ## Analysis

- **From**: client@example.com (known client)
- **Subject**: "Can you send me the invoice for January?"
- **Category**: Invoice Request
- **Priority**: High (revenue-related, 24h SLA applies)

### ## Proposed Response

**To**: client@example.com  
**Subject**: RE: Your January Invoice

Body:

"Hi there,

Thank you for reaching out! I've attached your January 2026 invoice for \$1,500.

Please let me know if you have any questions.

Best regards,  
[Your Name]"

**\*\*Attachment\*\***: /Vault/Invoices/2026-01\_ClientA.pdf

## Approval Required

- [ ] Verify invoice amount (\$1,500)
- [ ] Confirm client email is correct
- [ ] Check invoice template matches branding

Copy

#### Step 4: Move to Approval

When the response is ready:

- Create approval file in /Pending\_Approval/
- User reviews and moves to /Approved/ or /Rejected/
- When approved, Email MCP executes the send
- Log the action

Approval file format:

Copy

---  
type: approval\_request  
action: send\_email  
to: client@example.com  
subject: RE: Your January Invoice  
created: 2026-01-28T10:35:00Z  
expires: 2026-01-29T10:35:00Z  
status: pending  
---

## ## Message Preview

**\*\*To\*\***: client@example.com

**\*\*Subject\*\***: RE: Your January Invoice

Body:

"Hi there,

Thank you for reaching out! I've attached your January 2026 invoice for \$1,500.

Please let me know if you have any questions.

Best regards,

[Your Name]"

**\*\*Attachment\*\***: /Vault/Invoices/2026-01\_ClientA.pdf

## ## User Action

- **\*\*To send\*\***: Move to `/Approved/EMAIL\_invoice\_clienta.md`

- **\*\*To reject\*\***: Move to `/Rejected/EMAIL\_invoice\_clienta.md`

Copy

# Special Cases

## New Contact (Not in Company\_Handbook)

**Always** create an approval request before sending:

Copy

---

type: approval\_request

action: send\_email

to: nevendora@newcompany.com

subject: Inquiry about your services

status: pending

---

This is a NEW CONTACT. Please review before sending.

## Bulk Recipients

Flag for review if sending to multiple recipients:

Copy

recipients:

- person1@company.com
- person2@company.com
- person3@company.com

status: requires\_approval (bulk send)

## Legal/Sensitive Topics

Move immediately to /Pending\_Approval/ with ALWAYS\_REQUIRE\_APPROVAL

flag:

Copy

---

type: approval\_request

action: send\_email

to: lawyer@firmname.com

always\_require\_approval: true

sensitive: contract\_related

---

## Old Unread (> 48h old)

Check if still relevant before responding:

Copy

## Note

This email is 72 hours old. Check if the issue is still pending:

- [ ] Still relevant? Proceed

- [ ] Already resolved? Archive
- [ ] Waiting on us? Urgent response needed

## Draft Response Template

Always follow this format when drafting:

Copy

**\*\*To\*\***: \$FROM

**\*\*Subject\*\***: RE: \$SUBJECT

**\*\*Template\*\***: [Choose from Company\_*Handbook*]

**\*\*Body\*\***:

*[Personalized response following tone guidelines]*

*[Signature per handbook]*

**\*\*Attachments\*\***:

- *[If needed]*

**\*\*Approval Required\*\***: Yes/No

See reference.md for Gmail API details and examples/ for response patterns organized by category.

---

### Skill 3: Financial Auditor (CEO Briefing)

**\*\*Purpose\*\***: Generate the "Monday Morning CEO Briefing" — a proactive analysis of your business health.

**\*\*File Location\*\***: ``.claude/skills/financial-auditor/SKILL.md``

```yaml`

`name: financial-auditor`

description: Generate Monday Morning CEO Briefing by analyzing revenue, tasks completed, and bottlenecks from the past week. Identify cost optimization opportunities. Use weekly as a scheduled task.

disable-model-invocation: false

allowed-tools: Read, Write, Grep, Glob

Financial Auditor Skill

This skill generates the **"Monday Morning CEO Briefing"** — a proactive analysis showing revenue, completed work, bottlenecks, and cost-saving opportunities.

Trigger: Every Sunday 11pm (scheduled by Orchestrator)

Outputs: `/Briefings/YYYY-MM-DD\_Monday\_Briefing.md`

Weekly Audit Process

Input Data

Gather data from these vault files:

1. **`/Business\_Goals.md`** — Target metrics and objectives
2. **`/Accounting/Current\_Month.md`** — Transaction ledger
3. **`/Tasks/Done/`** — All completed tasks this week (timestamp-filtered)
4. **`/Pending\_Approval/`** — Stalled approvals > 24h old

Process: Five-Step Analysis

Step 1: Calculate Weekly Revenue

python

Pseudo-code for what you do:

week_start = last_monday

week_end = today

transactions = read("/Accounting/Current_Month.md")

```
revenue = sum([
    t.amount for t in transactions
    if t.date >= week_start
    and t.date <= week_end
    and t.type == "invoice_paid"
])
```

mtd_revenue = sum([

```

    t.amount for t in transactions
    if t.date.month == current_month
    and t.type == "invoice_paid"
])

target = read_from_business_goals("monthly_revenue_target")
percent_to_target = (mtd_revenue / target) * 100

```

Output example:

- This Week: \$2,450
- MTD (Month-to-Date): \$4,500
- Monthly Target: \$10,000
- Progress: 45% of target
- Trend: On track

Step 2: Analyze Completed Tasks

Copy

```

# Count and categorize work completed

tasks_done = read_files("/Tasks/Done/")
this_week = [t for t in tasks_done if t.timestamp >= week_start]

by_category = {
    "client_work": len([t for t in this_week if "invoice" in t.tags]),
    "admin": len([t for t in this_week if "admin" in t.tags]),
    "projects": len([t for t in this_week if "project" in t.tags]),
}

# Calculate actual vs estimated time
for task in this_week:
    delay = task.actual_time - task.estimated_time
    if delay > 0:
        bottlenecks.append({
            "task": task.name,
            "expected": task.estimated_time,
            "actual": task.actual_time,
            "delay_hours": delay
        })

```

Output example:

Completed Tasks:

- [x] Client A invoice sent and paid (\$1,500)
- [x] Project Alpha milestone 2 delivered
- [x] Weekly social media posts scheduled (5 posts)

Time Analysis:

- Task: "Client B proposal" - Expected 2 days, took 5 days (+3 days delay)
- Task: "Invoice generation" - Expected 30 min, took 15 min (-15 min ahead!)

Step 3: Identify Bottlenecks

Copy

Find patterns in delays

```
bottleneck_tasks = [t for t in this_week if t.delay_hours > 2]
recurring_blockers = []
```

```
for blocker in bottleneck_tasks:
    # Check if this blocker appeared in previous weeks
    if blocker.name in previous_bottleneck_log:
        recurring_blockers.append(blocker.name)
```

Calculate impact

```
total_delay_hours = sum([b.delay_hours for b in bottleneck_tasks])
```

Output example:

Bottlenecks This Week:

Task	Expected	Actual	Delay	Root Cause
Client B proposal	2 days	5 days	+3 days	Waiting for client feedback
Project Alpha design	1 day	2 days	+1 day	Unclear requirements

Recurring Issue:

- "Waiting for client feedback" blocked 8 hours of work (33% of delays)

Step 4: Find Cost Savings

Copy

```
# Analyze subscription spend
```

```
transactions = read("/Accounting/Current_Month.md")
subscriptions = {}
```

```
# Use pattern matching to identify subscriptions
```

```
PATTERNS = {
    'netflix.com': 'Entertainment',
    'slack.com': 'Productivity',
    'notion.so': 'Knowledge Mgmt',
    'adobe.com': 'Design Tools',
    'stripe.com': 'Payment Processing',
}
```

```
for trans in transactions:
    for pattern, category in PATTERNS.items():
        if pattern in trans.description.lower():
            if pattern not in subscriptions:
                subscriptions[pattern] = {
                    'name': category,
                    'amount': 0,
                    'count': 0,
                    'last_use': None
                }
            subscriptions[pattern]['amount'] += trans.amount
            subscriptions[pattern]['count'] += 1
```

```
# Check for optimization opportunities
```

```
candidates_for_cancellation = []
```

```
for sub_name, sub_data in subscriptions.items():
    days_since_last_use = (today - sub_data['last_use']).days

    if days_since_last_use > 30:
        candidates_for_cancellation.append({
            'name': sub_data['name'],
            'cost': sub_data['amount'],
            'unused_days': days_since_last_use
        })
```

Copy

Output example:

Cost Optimization Opportunities:

1. Notion: \$15/month

- No team activity in 45 days
- Recommendation: Cancel
- Monthly savings: \$15
- **ACTION**: Move to /Pending_Approval/CANCEL_NOTION.md

2. Adobe Creative Cloud: \$54.99/month

- 20% price increase from last month
- Recommendation: Review alternatives
- Check: Can use free design tools instead?

3. Slack: \$12.50/month

- Active usage (multiple users, daily)
- Keep subscribed

Step 5: Generate Briefing

Write comprehensive briefing to /Briefings/YYYY-MM-DD_Monday_Briefing.md:

Copy

Monday Morning CEO Briefing

****Period****: December 30, 2025 — January 5, 2026

****Generated****: January 6, 2026 at 7:00 AM

Executive Summary

Strong week with revenue ahead of target. One bottleneck identified in client delivery.

Revenue

- ****This Week****: \$2,450 
- ****MTD (Month-to-Date)****: \$4,500
- ****Monthly Target****: \$10,000
- ****Progress****: 45% of target
- ****Trend****: On track (+5% vs last week)

Completed Tasks

- ☒ Client A invoice sent and paid (\$1,500)
- ☒ Project Alpha milestone 2 delivered
- ☒ Weekly social media posts scheduled (5 posts)
- ☒ Email templates updated
- ☒ Two new client onboarding calls completed

****Work Velocity****: 5 projects completed (avg 8 hours each = 40 hours equivalent)

Bottlenecks

Task	Expected	Actual	Delay	Root Cause
Client B proposal	2 days	5 days	+3 days	Waiting for client feedback

****Analysis****: The Client B proposal was blocked by a missing design specification. Recommend clarifying requirements upfront before starting next projects.

****Recommendation****: Implement "Design Review" checkpoint before starting client design work.

Proactive Suggestions

Cost Optimization

- ****Notion****: No activity in 45 days. Cost: \$15/month → ****RECOMMEND: Cancel****
 - Action: Created `/Pending_Approval/CANCEL_NOTION.md`
- ****Adobe Creative Cloud****: 20% price increase. Cost: \$54.99/month
 - Analysis: Can we use free alternatives?
 - Action: Research Figma, Canva as alternatives

Upcoming Deadlines

- ****Project Alpha****: Final delivery in 9 days (Jan 15)
- ****Quarterly tax prep****: 25 days away (Jan 31)
- ****Q1 Planning****: 3 days away (Jan 9)

Team Insights

- ****Fastest completed task****: Invoice processing (avg 15 min)
- ****Most delayed****: Client proposals (avg +2 days)
- ****Success metric****: 100% of invoices paid on time this week

****Recommendation****: Automate more invoice steps. Consider template-based proposals to reduce turnaround on Client B type work.

Generated by AI Employee v0.1 on 2026-01-06 at 07:00:00Z

Copy

Subscription Pattern Matching

Use this mapping to categorize transactions (in Python):

Copy

```
SUBSCRIPTION_PATTERNS = {
    'netflix.com': ('Entertainment', '$9.99'),
    'spotify.com': ('Entertainment', '$12.99'),
    'slack.com': ('Productivity', '$12.50'),
    'notion.so': ('Knowledge Mgmt', '$15.00'),
    'openai.com': ('AI Services', 'variable'),
    'stripe.com': ('Payment Processing', 'variable'),
    'adobe.com': ('Design Tools', '$54.99'),
    'figma.com': ('Design Tools', '$12.00'),
    'github.com': ('Development', '$4.00'),
    'aws.amazon.com': ('Cloud', 'variable'),
}

def categorize_transaction(description):
    for pattern, (category, cost) in SUBSCRIPTION_PATTERNS.items():
        if pattern in description.lower():
            return category, cost
    return 'Other', 'unknown'
```

Error Handling

- **Missing Accounting data:** Note gap and estimate from last available data
- **No completed tasks:** Flag as low-activity week (investigate why)
- **Outdated Business_Goals:** Use last known targets with warning
- **No transactions:** Report \$0 revenue with investigation recommendation

Scheduling

This skill runs automatically **every Sunday at 11:00 PM** via the Orchestrator:

Copy

```
# In orchestrator.py
schedule.every().sunday.at("23:00").do(
    run_skill_with_task,
    skill='financial-auditor',
    task_description='Generate Monday Morning CEO Briefing'
)
```

See reference.md for full accounting taxonomy and examples/ for sample briefings from real use.

Skill 4: Approval Workflow Manager

****Purpose****: Implement human-in-the-loop (HITL) safety checks for sensitive actions.

****File Location****: ``.claude/skills/approval-workflow/SKILL.md``

````yaml`

---

```
name: approval-workflow
description: Manage human-in-the-loop approval workflow. Monitor /Pending_Approval/ folder,
present items for user review, handle approvals and rejections, and update the audit log.
disable-model-invocation: false
allowed-tools: Read, Write, Glob
```

---

# Approval Workflow Manager Skill

This skill implements **\*\*human-in-the-loop (HITL)\*\*** safety checks. Certain actions require explicit human approval before execution.

## Core Principle

**\*\*AI acts with permission, not autonomy, for sensitive operations.\*\***

## ## Approval Categories & Thresholds

### ### Auto-Approve (No HITL Required)

These actions don't need approval:

- Email to **\*\*known contacts\*\*** (in Company\_Handbook)
- Payments **\*\*< \$50\*\*** (recurring subscriptions)
- Social media posts (scheduled, non-reply)
- File operations (create/read, not delete)
- Bulk email **\*\*to known contacts only\*\***

**\*\*Example\*\***: Sending \$8.99 monthly Slack payment → auto-approve

### ### Always Require Approval

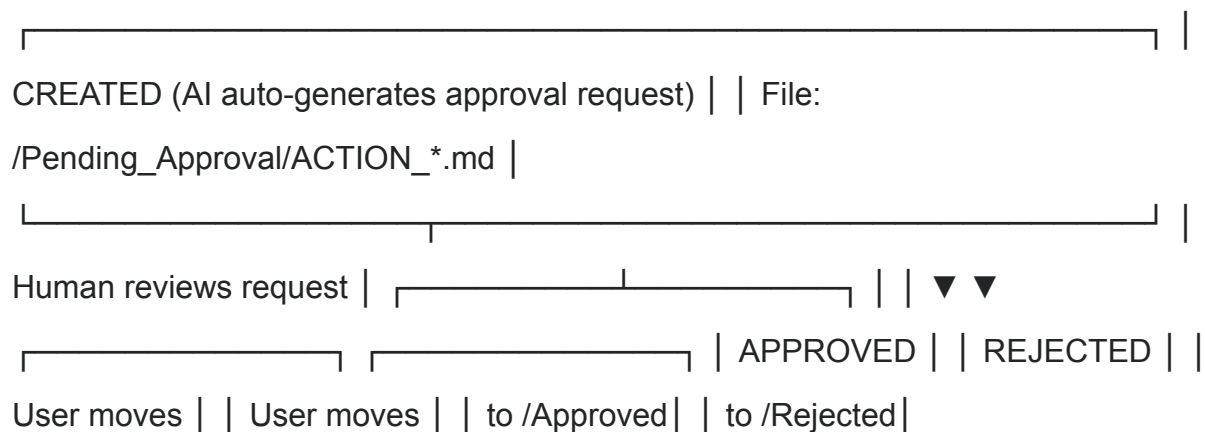
These actions **\*\*always\*\*** need explicit approval:

- **\*\*Payments to new recipients\*\*** (not in vendor list)
- **\*\*Payments > \$100\*\***
- **\*\*Email to new contacts\*\*** (not in Company\_Handbook)
- **\*\*Social media replies/DMs\*\*** (interactive, unpredictable)
- **\*\*Delete operations\*\*** (irreversible)
- **\*\*External API calls with side effects\*\***
- **\*\*Bulk email sends\*\*** (even if to known contacts)

**\*\*Example\*\***: Paying a new vendor \$250 → require approval

## ## Approval File Lifecycle

The approval workflow follows this state machine:



```

┌──────────┐ ┌──────────┐ | | ▼ ▼
└────────────────────────────────┘ | Orchestrator picks up file | | and
either executes or logs | ┌────────────────────────────────┘ | Logs
action to /Logs/YYYY-MM-DD.json | ▼
┌────────────────────────────────┘ | Move to /Done/ | | (archive after
execution) | ┌────────────────────────────────┘

```

### ## Approval Request Format

Every approval request must include these fields:

```

```yaml
---
type: approval_request
action: send_email | make_payment | post_social | delete_file | cancel_subscription
created: <ISO timestamp>
expires: <ISO timestamp + 24h>
status: pending
priority: high | normal | low
---

```

Details

[Action-specific details - see below for templates]

User Action

- **To Approve**: Move file to `/Approved/`
- **To Reject**: Move file to `/Rejected/`

Approval Request Templates

Email Approval (New Contact)

File: /Pending_Approval/EMAIL_new_vendor_inquiry.md

Copy

type: approval_request
action: send_email
to: vendor@example.com
subject: Inquiry about your services
created: 2026-01-28T10:30:00Z
expires: 2026-01-29T10:30:00Z
status: pending
priority: normal

Message Details

****To****: vendor@example.com
****Subject****: Inquiry about your services

****Body****:

Hi there,

I came across your software and think it might solve our reporting needs. Could you share pricing and implementation timeline?

Thanks, [Your Name]

Analysis

- ****Contact Type****: NEW (not in Company_Handbook)
- ****Action****: Email to new recipient requires approval
- ****Risk****: Low (no financial commitment)

User Decision

- ✓ ****Send****: Move to `/Approved/EMAIL\_new\_vendor\_inquiry.md`
- ✗ ****Don't Send****: Move to `/Rejected/EMAIL\_new\_vendor\_inquiry.md`

Payment Approval (New Recipient, > \$100)

File: /Pending_Approval/PAYMENT_new_vendor_invoice_2026-01-28.md

Copy

type: approval_request
action: make_payment
amount: 250.00
recipient: newvendor@example.com
reference_invoice: INV-5678
created: 2026-01-28T11:00:00Z
expires: 2026-01-29T11:00:00Z
status: pending
priority: high

Payment Details

- **Amount**: \$250.00
- **Recipient**: newvendor@example.com
- **Recipient Type**: NEW (not in approved vendor list)
- **Invoice**: INV-5678
- **Description**: Software licensing setup fee
- **Due**: 2026-01-30

Analysis

- **Approval Required**: Yes (new recipient + amount > \$100)
- **Recipient Verification**: Email domain matches invoice
- **Amount Check**: \$250 is reasonable for setup fee per industry norms

User Decision

- ✓ **Approve Payment**: Move to `/Approved/PAYMENT\_new\_vendor\_invoice\_2026-01-28.md`
- ✗ **Reject Payment**: Move to `/Rejected/PAYMENT\_new\_vendor\_invoice\_2026-01-28.md`

Copy

Social Media Approval (Reply/DM)

File: /Pending_Approval/SOCIAL_twitter_reply_client.md

Copy

type: approval_request
action: post_social
platform: twitter
type: reply
in_reply_to: tweet_id_123
created: 2026-01-28T14:20:00Z
expires: 2026-01-28T16:20:00Z
status: pending
priority: high

Tweet Details

Platform: Twitter/X
Type: Reply to @clientname
Audience: Public (followers + thread participants)

Proposed Tweet:

@clientname Thanks for the question! Here's how we approach that problem:
[explanation]. Happy to discuss further.

Analysis

- **Approval Required**: Yes (replies are interactive, can lead to conversation)
- **Risk Level**: Medium (public, client-facing)
- **Tone**: Professional, helpful

User Decision

- ✓ **Post**: Move to `/Approved/SOCIAL\_twitter\_reply\_client.md`
- ✗ **Don't Post**: Move to `/Rejected/SOCIAL\_twitter\_reply\_client.md`
- 📝 **Edit First**: Reject and provide new text for next attempt

Handling Approvals

When user moves file to /Approved/:

Step 1: Validate

Check that:

- Approval request hasn't expired
- File is properly formatted
- Action is still relevant

Copy

If expired:

```
if datetime.now() > expires_timestamp:
    move_to_done("APPROVAL_EXPIRED")
    log_action("approval_expired", "Action no longer valid")
    return
```

Step 2: Execute

Trigger appropriate MCP handler based on action type:

Copy

```
action_type = frontmatter['action']

if action_type == 'send_email':
    execute_email_mcp(frontmatter)
elif action_type == 'make_payment':
    execute_payment_mcp(frontmatter)
elif action_type == 'post_social':
    execute_social_mcp(frontmatter)
elif action_type == 'cancel_subscription':
    execute_cancel_subscription_mcp(frontmatter)
```

Step 3: Log Success

Write to audit log:

Copy

```
{
  "timestamp": "2026-01-28T14:30:00Z",
  "action_type": "email_send",
  "actor": "claude_code",
  "target": "client@example.com",
  "parameters": {
    "subject": "Invoice #123",
    "body_preview": "Please find attached..."
  },
  "approval_status": "approved",
  "approved_by": "human_user",
  "result": "success",
  "mcp_server": "email-mcp",
  "execution_time_ms": 245
}
```

Step 4: Archive

Move to /Done/ with timestamp:

```
/Approved/EMAIL_invoice_clienta.md
→ /Done/EMAIL_invoice_clienta_2026-01-28T14-30-00.md
```

Handling Rejections

When user moves file to /Rejected/:

Step 1: Archive

Move to /Logs/REJECTED/ with timestamp:

```
/Rejected/PAYMENT_new_vendor.md
→ /Logs/REJECTED/PAYMENT_new_vendor_2026-01-28T14-35-00.md
```

Step 2: Log Rejection

Record in audit log:

Copy

```
{
  "timestamp": "2026-01-28T14:35:00Z",
  "action_type": "payment_proposed",
  "recipient": "vendor@example.com",
  "amount": 250.00,
  "reason": "Insufficient funds / Incorrect recipient / Timing wrong",
  "approval_status": "rejected_by_user",
  "result": "abandoned"
}
```

Step 3: Update Dashboard

Add to Dashboard.md:

Copy

Recent Actions

- [2026-01-28 14:35] ❌ Payment rejected: \$250 to newvendor@example.com
(Reason: Verify vendor legitimacy first)

Step 4: Don't Retry

Never automatically resubmit a rejected action. Require human intervention.

Approval Thresholds by Category

This table summarizes when approval is required:

Category	Threshold	Auto-Approve?	Notes
Email	New contact	NO	Always review first
Email	Known contact	YES	Fast-track
Email	Bulk (> 3 recipients)	NO	Always review
Payment	< \$50 recurring	YES	Subscriptions ok
Payment	\$50-\$500 new recipient	NO	Verify vendor
Payment	Any > \$500	NO	Always require
Social	Scheduled post	YES	Pre-written
Social	Reply/DM	NO	Unpredictable interaction

| - task_id: unique identifier | | - status: in_progress | | - current_iteration: 1 | | -
max_iterations: 10 |

└ | ▼

└ | Step 2: Claude Works on Task | | | Claude reads TASK file: | | 1. Understands
objective | | 2. Reads current progress | | 3. Takes next steps | | 4. Updates
progress in vault | | 5. Creates sub-files as needed (/Plans/, etc) |

└ | ▼

└ | Step 3: Check Completion | | | At end of iteration, Claude checks: | | "Is task
file moved to /Done/?" | | | Completion signal: File exists at | |
/Done/TASK_process_weekly_inbox_2026-01-28.md |

└ | ┌──────────┐ | | NO | | YES ▼ ▼ ┌──────────┐
┌──────────┐ | Still Working | | Task Complete | | | Exit SUCCESS | |
Check: | ┌──────────┐ | - Iterations | | - Max reached? |
┌──────────┐ | NO | YES ┌──────────┐ | | ▼ ▼ Continue
Abort with Looping MAX_ITERATIONS Error

Task State File Format

The Orchestrator creates a task state file like this:

```
``yaml
---
type: task
task_id: process_weekly_inbox
created: 2026-01-28T08:00:00Z
status: in_progress
```

```
max_iterations: 10
current_iteration: 1
started_at: 2026-01-28T08:00:00Z
---
```

Objective

Process all items in /Needs_Action/ and move completed items to /Done/

Expected Steps

1. List all files in /Needs_Action/
2. Process each file by type (email, whatsapp, file drop)
3. Create plans/approvals as needed
4. Move processed files to /Done/
5. Update Dashboard.md with summary
6. **COMPLETION:** All /Needs_Action items processed

Progress

****Iteration 1**** (current):

- [] List /Needs_Action items
- [] Start processing first item

Iteration Pattern Example

How Claude progresses through iterations:

Iteration 1: Discover and Start

Copy

Progress

****Iteration 1**** (current):

- [x] List /Needs_Action items
Found:
 - EMAIL_abc123.md (from client@example.com, invoice request)
 - WHATSAPP_msg001.md (from Client B, urgent quote request)
 - FILE_report.pdf (document drop)
- [x] Start processing first item

Processing: EMAIL_abc123 (invoice request)
Next iteration: Complete plan and request approval

At end of iteration, Claude checks: "Is task in /Done/?" → No, continue looping

Iteration 2: Process First Item

Copy

Progress

****Iteration 2**** (current):

- [x] EMAIL_abc123: Created PLAN_invoice_client_abc123.md
- [x] EMAIL_abc123: Moved to APPROVAL_REQUEST

User still needs to approve, so this item is blocked.

- [x] Start processing second item
Processing: WHATSAPP_msg001 (quote request)
Next iteration: Draft response

At end of iteration, Claude checks: "Is task in /Done/?" → No, continue looping

Iteration 3: Progress to Completion

Copy

Progress

****Iteration 3**** (current):

- [x] WHATSAPP_msg001: Created PLAN_quote_response.md
- [x] WHATSAPP_msg001: Moved to APPROVAL_REQUEST

- [x] Start processing third item
Processing: FILE_report.pdf (document drop)

This is a system-generated report. File appropriately.

- [x] All /Needs_Action items have been processed!
Summary:
- 3 items processed

- 3 approval requests created
- 0 items auto-completed
- Awaiting user approvals to proceed

TASK COMPLETE: All inbox items triaged and planned.

At end of iteration, Claude checks: "Is task in /Done/?" → Claude moves task to /Done/TASK_process_weekly_inbox_2026-01-28.md → Loop detects completion and exits

Iteration Exit Conditions

Claude loops while:

- `current_iteration < max_iterations`
- Task file is in /Tasks/ (not yet moved to /Done/)

Claude exits when:

- ✓ Task file moves to /Done/ (SUCCESS)
- ✓ `current_iterations >= max_iterations` (TIMEOUT)
- ✓ Task file moves to /Logs/INCOMPLETE/ (ERROR)

Max Iterations Protection

Prevent infinite loops:

Copy

In orchestrator

MAX_ITERATIONS = 10

if `current_iteration > MAX_ITERATIONS`:

```
# Don't let it loop forever
move_to_done(task_id, "MAX_ITERATIONS_EXCEEDED")
log_error(f"Task {task_id} hit iteration limit")
send_alert("Claude loop exceeded max iterations")
exit_with_error()
```

Completion Strategies

Strategy 1: File Movement (Recommended)

Task is complete when file moves to /Done/:

Copy

```
# Orchestrator checks after each iteration
if Path(f"/Vault/Done/TASK_{TASK_ID}.md").exists():
    print("✓ Task complete - file found in /Done/")
    exit(0)
else:
    print("⌚ Task still in progress - re-injecting prompt")
    re_inject_prompt_with_current_state()
```

Claude writes at the end of an iteration:

Copy

```
# Claude's final action in iteration
task_file = Path("/Vault/Tasks/TASK_xyz.md")
done_file = Path("/Vault/Done/TASK_xyz.md")

# When task is complete:
task_file.rename(done_file)
print("✓ Task complete - moved to /Done/")
```

Strategy 2: Marker in File (Alternative)

Task includes explicit completion marker:

Copy

```
---
type: task
status: complete
completion_marker: INVOICE_BATCH_COMPLETE_V2
completed_at: 2026-01-28T09:30:00Z
---
```

Orchestrator greps for marker:

Copy

```
# Orchestrator checks
if grep -q "completion_marker: INVOICE_BATCH_COMPLETE" /Vault/TASK_xyz.md; then
  echo "✓ Completion marker found"
  exit 0
fi
```

Recommendation: Use Strategy 1 (file movement) — it's more reliable and aligns with the vault's state machine design.

Usage Example

Terminal command (run by Orchestrator.py):

Copy

```
claude --cwd /path/to/vault \
  --skill ralph-wiggum-loop \
  "Process all /Needs_Action items until complete, then move task to /Done/"
```

What happens:

- Orchestrator.py creates /Tasks/TASK_inbox_2026-01-28.md
- Calls Claude with ralph-wiggum-loop skill
- Claude iterates until task file moves to /Done/

- Orchestrator detects completion and returns success

Error Cases

If task never moves to /Done/

Iteration 1: Processing items...

Iteration 2: Still processing...

...

Iteration 10: MAX_ITERATIONS reached

Action taken:

1. Log final state to /Logs/TASK_xyz_final_state.md
2. Move task to /Logs/INCOMPLETE/TASK_xyz.md
3. Alert human: "Task reached iteration limit"
4. Exit with error code 1

If task is in error state

Quarantine and alert:

Copy

```
if error_detected():  
    move_task_to("/Logs/ERRORS/")  
    log_error_with_context()  
    alert_human("Task encountered error")  
    exit(1)
```

Real-World Example: Weekly Financial Audit

Task: "Generate Monday Morning CEO Briefing"

Copy

type: task

task_id: financial_audit_2026-01-27

created: 2026-01-27T23:00:00Z
status: in_progress
max_iterations: 5 # This task is usually quick

Objective

Generate comprehensive CEO Briefing for the week of Dec 30 - Jan 5.
Include revenue, completed tasks, bottlenecks, and cost savings.

Progress

Iteration 1:

- [x] Read Business_Goals.md to get metrics
- [x] Scan Accounting/Current_Month.md for transactions
- [x] Calculate weekly revenue: \$2,450
- [x] Next: Analyze completed tasks

Iteration 2:

- [x] Read all files from /Tasks/Done/ from past week
- [x] Count by category (client work: 3, admin: 2, projects: 1)
- [x] Identify bottlenecks (Client B proposal: +3 day delay)
- [x] Next: Scan for cost optimization

Iteration 3:

- [x] Pattern-match subscription transactions
- [x] Found candidates: Notion (\$15, unused 45 days), Adobe (\$54.99, 20% increase)
- [x] Generated CEO Briefing file
- [x] COMPLETE: Moved task to /Done/

Copy

See reference.md for implementation details and common patterns.

Watcher System Design

Overview: The Sensory System

Watchers are lightweight Python daemon processes that continuously monitor external sources (Gmail, WhatsApp, bank APIs) and create action files for Claude to process. They solve the fundamental problem: **Claude can't listen to the internet 24/7**, but watchers can.

Base Watcher Architecture

****File: `watchers/base\_watcher.py`****

All Watchers **inherit** from `BaseWatcher`, which provides standard patterns for:

- Polling external sources
- Creating markdown files **with** proper formatting
- Deduplication (don't process same item twice)
- Error handling and logging

```
``python
import time
import logging
from pathlib import Path
from abc import ABC, abstractmethod
import json
from datetime import datetime

class BaseWatcher(ABC):
    """
    Base class for all Watchers. A Watcher is a lightweight daemon that monitors
    external sources (Gmail, WhatsApp, bank accounts) and writes .md files to
    /Needs_Action/ for Claude to process.

    These files trigger Claude Code with the appropriate skills:
    - EMAIL_* files → gmail-processor skill
    - WHATSAPP_* files → whatsapp-handler skill
    - TRANSACTION_* files → financial-auditor skill
    - FILE_* files → vault-operations skill
    """

    def __init__(self, vault_path: str, check_interval: int = 60):
        """
        Initialize a Watcher.

        Args:
            vault_path: Path to Obsidian vault root
            check_interval: Seconds between polling external source (default 60)
        """
        self.vault_path = Path(vault_path)
        self.needs_action = self.vault_path / 'Needs_Action'
        self.check_interval = check_interval
        self.logger = logging.getLogger(self.__class__.__name__)
        self.processed_items = set() # Track what we've already reported
```

```

# Ensure directories exist
self.needs_action.mkdir(parents=True, exist_ok=True)

@abstractmethod
def check_for_updates(self) -> list:
    """
    Check external source for new items.

    Returns:
        List of new items from source (subclass-specific format)
    """
    pass

@abstractmethod
def create_action_file(self, item) -> Path:
    """
    Create .md file in Needs_Action folder for Claude to process.

    Args:
        item: Item from check_for_updates()

    Returns:
        Path to created markdown file
    """
    pass

def run(self):
    """
    Main loop - run continuously in background.
    Polls check_for_updates() every check_interval seconds.
    """
    self.logger.info(f'Starting {self.__class__.__name__}')
    while True:
        try:
            items = self.check_for_updates()
            for item in items:
                self.create_action_file(item)
                self.logger.info(f'Created action file for: {item}')
            except Exception as e:
                self.logger.error(f'Error in check loop: {e}', exc_info=True)

            time.sleep(self.check_interval)

def _create_markdown_file(self, filename: str, frontmatter: dict, content: str) -> Path:
    """
    Helper to create properly formatted .md file with YAML frontmatter.

```

Args:

filename: Name of file (e.g., 'EMAIL_123.md')

frontmatter: Dict of YAML fields

content: Markdown content

Returns:

Path to created file

```
"""
```

```
# Build YAML frontmatter
```

```
yaml_lines = ['---']
```

```
for key, value in frontmatter.items():
```

```
    if isinstance(value, str):
```

```
        yaml_lines.append(f'{key}: {value}')
```

```
    else:
```

```
        yaml_lines.append(f'{key}: {json.dumps(value)}')
```

```
yaml_lines.append('---')
```

```
yaml_section = '\n'.join(yaml_lines)
```

```
# Combine with content
```

```
full_content = f'{yaml_section}\n\n{content}'
```

```
filepath = self.needs_action / filename
```

```
filepath.write_text(full_content)
```

```
return filepath
```

Gmail Watcher Implementation

File: watchers/gmail_watcher.py

Monitors Gmail for unread important messages and creates EMAIL_*.md files.

Copy

```
from base_watcher import BaseWatcher
```

```
from google.oauth2.credentials import Credentials
```

```
from googleapiclient.discovery import build
```

```
from datetime import datetime
```

```
import os
```

```
class GmailWatcher(BaseWatcher):
```

```
    """
```

```
    Monitors Gmail for unread important messages.
```

Creates EMAIL_* files that trigger the gmail-processor skill.

"""

```
def __init__(self, vault_path: str, credentials_file: str, check_interval: int = 120):
```

"""

Initialize Gmail Watcher.

Args:

 vault_path: Path to Obsidian vault root

 credentials_file: Path to Google OAuth credentials JSON

 check_interval: Check interval in seconds (default 120 = 2 minutes)

"""

```
super().__init__(vault_path, check_interval)
```

```
self.creds = Credentials.from_authorized_user_file(credentials_file)
```

```
self.service = build('gmail', 'v1', credentials=self.creds)
```

```
self.processed_ids = set()
```

```
def check_for_updates(self) -> list:
```

 """Check for unread important emails."""

```
results = self.service.users().messages().list(
```

 userId='me',

 q='is:unread is:important',

 maxResults=10

```
).execute()
```

```
messages = results.get('messages', [])
```

```
new_messages = [m for m in messages if m['id'] not in self.processed_ids]
```

```
return new_messages
```

```
def create_action_file(self, message) -> Path:
```

 """Create EMAIL_*.md file with email details."""

```
msg_data = self.service.users().messages().get(
```

 userId='me',

 id=message['id'],

 format='full'

```
).execute()
```

```
headers = {h['name']: h['value'] for h in msg_data['payload']['headers']}
```

Determine priority based on keywords

```
subject = headers.get('Subject', 'No Subject').lower()
```

```
snippet = msg_data.get('snippet', '').lower()
```

```
text = f"{subject} {snippet}"
```

```
priority = 'high' if any(kw in text for kw in ['urgent', 'asap', 'invoice']) else 'normal'
```

```

frontmatter = {
    'type': 'email',
    'from': headers.get('From', 'Unknown'),
    'subject': headers.get('Subject', 'No Subject'),
    'received': datetime.now().isoformat(),
    'message_id': message['id'],
    'priority': priority,
    'status': 'pending'
}

content = f"""## Email Content

{msg_data.get('snippet', 'No preview available')}

## Suggested Actions

- [ ] Analyze and categorize
- [ ] Draft response (if needed)
- [ ] Create approval request
- [ ] Archive after processing
"""

filename = f"EMAIL_{message['id']}.md"
filepath = self._create_markdown_file(filename, frontmatter, content)
self.processed_ids.add(message['id'])

return filepath

if __name__ == '__main__':
    import logging
    logging.basicConfig(level=logging.INFO)

    vault_path = os.getenv('VAULT_PATH', '/path/to/AI_Employee_Vault')
    creds_file = os.getenv('GMAIL_CREDS', '/path/to/credentials.json')

    watcher = GmailWatcher(vault_path, creds_file)
    watcher.run()

```

Copy

WhatsApp Watcher (Playwright)

File: watchers/whatsapp_watcher.py

Monitors WhatsApp Web using Playwright for browser automation. Creates
WHATSAPP_*.md files.

Copy

```
from base_watcher import BaseWatcher
from playwright.sync_api import sync_playwright
from pathlib import Path
from datetime import datetime
import os

class WhatsAppWatcher(BaseWatcher):
    """
    Monitors WhatsApp Web for urgent messages (keywords: invoice, payment, urgent, etc).
    Uses Playwright for browser automation.
    Creates WHATSAPP_* files that trigger whatsapp-handler skill.

    Note: WhatsApp Web monitoring must comply with WhatsApp Terms of Service.
    """

    def __init__(self, vault_path: str, session_path: str, check_interval: int = 30):
        """
        Initialize WhatsApp Watcher.

        Args:
            vault_path: Path to Obsidian vault root
            session_path: Path to persisted browser session
            check_interval: Check interval in seconds (default 30)
        """
        super().__init__(vault_path, check_interval)
        self.session_path = Path(session_path)
        self.keywords = ['invoice', 'payment', 'urgent', 'asap', 'help', 'order', 'quote']
        self.last_seen_ids = set()

    def check_for_updates(self) -> list:
        """Check for unread WhatsApp messages containing keywords."""
        messages = []

        try:
            with sync_playwright() as p:
                browser = p.chromium.launch_persistent_context(
                    str(self.session_path),
                    headless=True
```

```

)

if not browser.pages:
    self.logger.warning("No pages in persistent context")
    return messages

page = browser.pages[0]
page.goto('https://web.whatsapp.com', timeout=30000)

try:
    page.wait_for_selector('[data-testid="chat-list"]', timeout=10000)
except:
    self.logger.warning("Could not find chat list")
    browser.close()
    return messages

# Find unread chats
unread_chats = page.query_selector_all('[aria-label*="unread"]')

for chat in unread_chats:
    try:
        text = chat.inner_text().lower()
        if any(kw in text for kw in self.keywords):
            # Extract sender and message preview
            sender_elem = chat.query_selector('[data-testid="cell-frame-title"]')
            sender = sender_elem.inner_text() if sender_elem else "Unknown"

            msg_id = f"{sender}_{datetime.now().timestamp()}"
            if msg_id not in self.last_seen_ids:
                messages.append({
                    'sender': sender,
                    'preview': text[:200],
                    'id': msg_id,
                    'timestamp': datetime.now().isoformat()
                })
                self.last_seen_ids.add(msg_id)
    except Exception as e:
        self.logger.warning(f"Error processing chat: {e}")

browser.close()

except Exception as e:
    self.logger.error(f"Error in WhatsApp watcher: {e}", exc_info=True)

return messages

```

```

def create_action_file(self, message) -> Path:
    """Create WHATSAPP_*.md file."""
    frontmatter = {
        'type': 'whatsapp_message',
        'from': message['sender'],
        'received': message['timestamp'],
        'message_id': message['id'],
        'priority': 'high',
        'status': 'pending'
    }

    content = f"""## Message Preview

{message['preview']}

## Suggested Actions

- [ ] Read full message
- [ ] Determine response type
- [ ] Draft reply
- [ ] Send (with approval if needed)
"""

    filename = f"WHATSAPP_{message['id']}.md"
    return self._create_markdown_file(filename, frontmatter, content)

if __name__ == '__main__':
    import logging
    logging.basicConfig(level=logging.INFO)

    vault_path = os.getenv('VAULT_PATH', '/path/to/AI_Employee_Vault')
    session_path = os.getenv('WHATSAPP_SESSION', '/path/to/whatsapp_session')

    watcher = WhatsAppWatcher(vault_path, session_path)
    watcher.run()

```

Copy

File System Watcher

File: `watchers/filesystem_watcher.py`

Monitors a local "drop folder" for file uploads. Creates FILE_* .md files.

Copy

```
from base_watcher import BaseWatcher
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
from pathlib import Path
from datetime import datetime
import shutil
import os

class DropFolderHandler(FileSystemEventHandler):
    """Handles file drops into monitoring folder."""

    def __init__(self, vault_path: str):
        self.needs_action = Path(vault_path) / 'Needs_Action'
        self.logger = logging.getLogger('DropFolderHandler')

    def on_created(self, event):
        """Handle new file in drop folder."""
        if event.is_directory:
            return

        try:
            source = Path(event.src_path)
            dest = self.needs_action / f'FILE_{source.name}'
            shutil.copy2(source, dest)

            # Create metadata file
            meta_path = dest.with_suffix('.md')
            content = f"""---
type: file_drop
original_name: {source.name}
size: {source.stat().st_size}
created: {datetime.now().isoformat()}
status: pending
---

### File Details

- **Name**: {source.name}
- **Size**: {source.stat().st_size} bytes
- **Type**: {source.suffix}

### Suggested Actions
```

- [] Review file contents
- [] Categorize (report, invoice, proposal, etc)
- [] File appropriately
- [] Archive after processing

```

"""
    meta_path.write_text(content)
    self.logger.info(f"File dropped: {source.name}")

except Exception as e:
    self.logger.error(f"Error processing file drop: {e}")

def monitor_drop_folder(vault_path: str, drop_folder: str):
    """Start monitoring a drop folder."""
    handler = DropFolderHandler(vault_path)
    observer = Observer()
    observer.schedule(handler, drop_folder, recursive=False)
    observer.start()

    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        observer.stop()
        observer.join()

if __name__ == '__main__':
    import logging
    import time
    logging.basicConfig(level=logging.INFO)

    vault_path = os.getenv('VAULT_PATH', '/path/to/Al_Employee_Vault')
    drop_folder = os.getenv('DROP_FOLDER', '/path/to/drop_folder')

    monitor_drop_folder(vault_path, drop_folder)

```

Copy

Orchestrator Architecture

Master Coordination Engine

The **Orchestrator** is the conductor of your AI Employee. It:

- **Schedules** Claude to run with appropriate skills
- **Monitors** the vault for task updates
- **Watches** for approvals and executes MCPs
- **Manages** process lifecycle
- **Logs** all actions for audit trail

File: orchestrator.py

Copy

```
#!/usr/bin/env python3
"""
```

Orchestrator.py - Master coordination engine for Personal AI Employee.

Responsibilities:

1. Schedule Claude to run with appropriate skills
2. Monitor vault for task updates
3. Watch for approvals and execute MCPs
4. Manage process lifecycle
5. Log all actions

```
"""
```

```
import subprocess
```

```
import time
```

```
import json
```

```
from pathlib import Path
```

```
from datetime import datetime, timedelta
```

```
import schedule
```

```
import logging
```

```
from typing import Optional
```

```
import os
```

```
logging.basicConfig(
```

```
    level=logging.INFO,
```

```
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
```

```
    handlers=[
```

```
        logging.FileHandler('/path/to/AI_Employee_Vault/Logs/orchestrator.log'),
```

```
        logging.StreamHandler()
```

```

    ]
)
logger = logging.getLogger('Orchestrator')

class PersonalAIOrchestrator:
    """Master orchestrator for Personal AI Employee."""

    def __init__(self, vault_path: str):
        """
        Initialize orchestrator.

        Args:
            vault_path: Path to Obsidian vault root
        """
        self.vault_path = Path(vault_path)
        self.needs_action = self.vault_path / 'Needs_Action'
        self.approved = self.vault_path / 'Approved'
        self.done = self.vault_path / 'Done'
        self.logs = self.vault_path / 'Logs'
        self.in_progress = self.vault_path / 'In_Progress'

        # Ensure directories exist
        for d in [self.needs_action, self.approved, self.done, self.logs, self.in_progress]:
            d.mkdir(parents=True, exist_ok=True)

    def schedule_tasks(self):
        """Set up daily/weekly scheduled tasks."""

        # Every Monday at 8 AM: Generate CEO Briefing
        schedule.every().monday.at("08:00").do(
            self.run_skill_with_task,
            skill='financial-auditor',
            task_description='Generate Monday Morning CEO Briefing for past week'
        )

        # Every 2 hours: Process inbox
        schedule.every(2).hours.do(
            self.run_skill_with_task,
            skill='vault-operations',
            task_description='Process /Needs_Action inbox items'
        )

        # Every 30 minutes: Check for approvals
        schedule.every(30).minutes.do(self.check_approvals)

    logger.info("Scheduled tasks initialized")

```

```

def run_skill_with_task(self, skill: str, task_description: str):
    """
    Run Claude with a specific skill using Ralph Wiggum loop pattern.

    This method:
    1. Creates a task state file
    2. Calls Claude Code with the skill
    3. Monitors for task completion
    4. Logs the result

    Args:
        skill: Skill name (e.g., 'financial-auditor', 'gmail-processor')
        task_description: Description of task for Claude
    """
    task_id = f"{skill}_{datetime.now().strftime('%Y%m%d_%H%M%S')}"
    task_file = self.vault_path / f"TASK_{task_id}.md"

    # Create task state file
    task_state = f"""---
type: task
task_id: {task_id}
created: {datetime.now().isoformat()}
skill: {skill}
status: in_progress
max_iterations: 10
current_iteration: 0
---

## Objective

{task_description}

## Status

Created at {datetime.now().isoformat()}

## Progress

(Claude will update this section as it works)
"""
    task_file.write_text(task_state)
    logger.info(f"Created task: {task_id}")

    # Call Claude with skill
    try:

```

```

cmd = [
    'claude',
    '--cwd', str(self.vault_path),
    '--skill', skill,
    f'Process this task: {task_description}'
]

result = subprocess.run(cmd, capture_output=True, text=True, timeout=600)

if result.returncode == 0:
    logger.info(f"Skill {skill} completed successfully")
    # Move task to done if not already there
    done_file = self.done / f"TASK_{task_id}.md"
    if task_file.exists():
        task_file.rename(done_file)
    self.log_action('skill_executed', skill, 'success')
else:
    logger.error(f"Skill {skill} failed: {result.stderr}")
    self.log_action('skill_executed', skill, 'failure', result.stderr)

except subprocess.TimeoutExpired:
    logger.error(f"Skill {skill} timed out after 10 minutes")
    self.log_action('skill_executed', skill, 'timeout')
except Exception as e:
    logger.error(f"Error running skill {skill}: {e}")
    self.log_action('skill_executed', skill, 'error', str(e))

def check_approvals(self):
    """
    Check for approved actions in /Approved/ folder and execute via MCP.

    When a user moves a file from /Pending_Approval/ to /Approved/,
    this method:
    1. Detects the approved file
    2. Parses its action type
    3. Calls appropriate MCP server
    4. Logs result
    5. Archives to /Done/
    """
    if not self.approved.exists():
        return

    approved_files = list(self.approved.glob("*.md"))
    for f in approved_files:
        try:
            content = f.read_text()

```

```

# Parse frontmatter to get action type
if content.startswith('---'):
    yaml_end = content.find('---', 3)
    yaml_section = content[4:yaml_end]

    action_type = None
    if 'action: send_email' in yaml_section:
        action_type = 'send_email'
    elif 'action: make_payment' in yaml_section:
        action_type = 'make_payment'
    elif 'action: post_social' in yaml_section:
        action_type = 'post_social'

    if action_type:
        success = self.execute_mcp(f, action_type)
        # Move to Done after execution
        timestamp = datetime.now().strftime('%Y-%m-%dT%H-%M-%S')
        f.rename(self.done / f'{f.stem}_{timestamp}.md')
        logger.info(f'{f} ✓' if success else '✗' Executed: {f.name})

except Exception as e:
    logger.error(f"Error processing approval {f.name}: {e}")

def execute_mcp(self, approval_file: Path, action_type: str) -> bool:
    """
    Execute action via appropriate MCP server.

    This is where you integrate with your MCP implementations
    for email, payments, social media, etc.

    Args:
        approval_file: Path to approval request file
        action_type: Type of action to execute

    Returns:
        True if successful, False otherwise
    """
    try:
        logger.info(f"Executing MCP action: {action_type} for {approval_file.name}")

        # Parse the approval file
        content = approval_file.read_text()
        yaml_section = content[content.find('---')+3:content.find('---', 3)]

        if action_type == 'send_email':
            return self._execute_email_send(yaml_section, content)

```

```

        elif action_type == 'make_payment':
            return self._execute_payment(yaml_section, content)
        elif action_type == 'post_social':
            return self._execute_social_post(yaml_section, content)

    return False

except Exception as e:
    logger.error(f"MCP execution error: {e}")
    return False

def _execute_email_send(self, yaml_section: str, content: str) -> bool:
    """Execute email sending via Email MCP."""
    # Extract email details from YAML
    # This would call your email-mcp server
    logger.info("Would execute email send")
    return True

def _execute_payment(self, yaml_section: str, content: str) -> bool:
    """Execute payment via Payment MCP."""
    # Extract payment details from YAML
    # This would call your payment-mcp server
    logger.info("Would execute payment")
    return True

def _execute_social_post(self, yaml_section: str, content: str) -> bool:
    """Execute social media post via Social MCP."""
    # Extract social details from YAML
    # This would call your social-mcp server
    logger.info("Would execute social post")
    return True

def log_action(self, action_type: str, detail: str, status: str, error: str = None):
    """
    Log all actions to audit trail.

    Args:
        action_type: Type of action (skill_executed, approval_processed, etc)
        detail: Details of the action
        status: success, failure, timeout, etc
        error: Error message if applicable
    """
    log_file = self.logs / f"{datetime.now().strftime('%Y-%m-%d')}.json"

    entry = {
        'timestamp': datetime.now().isoformat(),

```

```

        'action_type': action_type,
        'detail': detail,
        'status': status,
        'error': error
    }

    # Append to daily log
    entries = []
    if log_file.exists():
        entries = json.loads(log_file.read_text())

    entries.append(entry)
    log_file.write_text(json.dumps(entries, indent=2))

def start(self):
    """Main orchestrator loop - run continuously."""
    self.schedule_tasks()
    logger.info("=" * 60)
    logger.info("Personal AI Employee Orchestrator starting")
    logger.info("=" * 60)

    while True:
        schedule.run_pending()
        time.sleep(60)

if __name__ == '__main__':
    vault_path = os.getenv('VAULT_PATH', '/path/to/AI_Employee_Vault')
    orchestrator = PersonalAIOrchestrator(vault_path)
    orchestrator.start()

```

Copy

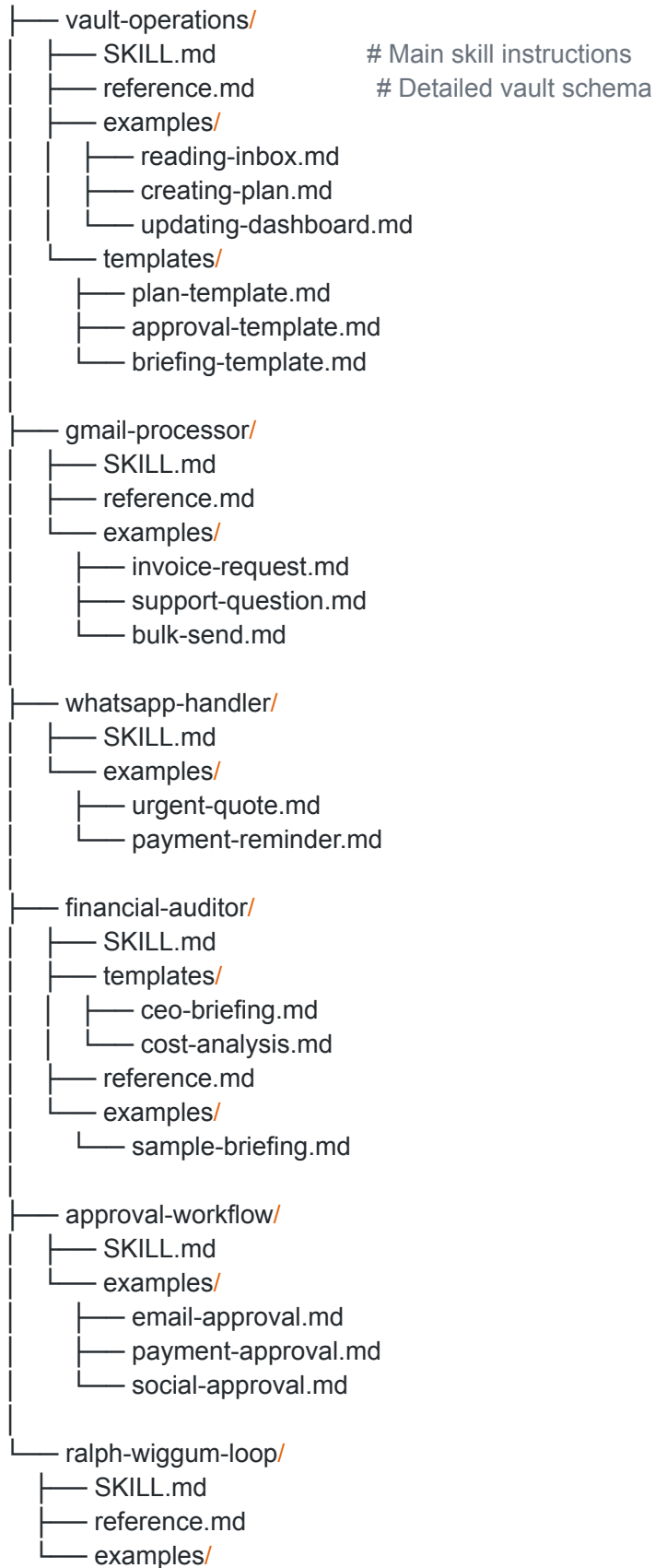
Project Structure & Setup

Complete Directory Layout

```

ai_employee_project/
├── .claude/
│   └── skills/                # Claude Code skills directory

```



└─ task-completion.md	
└─ watchers/	# Sentinel scripts (daemon processes)
└─ base_watcher.py	# Base class for all watchers
└─ gmail_watcher.py	# Gmail monitoring
└─ whatsapp_watcher.py	# WhatsApp Web monitoring
└─ filesystem_watcher.py	# File drop monitoring
└─ __init__.py	
└─ orchestrator.py	# Master coordination engine
└─ watchdog.py	# Process health monitor
└─ AI_Employee_Vault/	# Obsidian vault root
└─ Needs_Action/	# Inbox (fresh items from Watchers)
└─ In_Progress/	
└─ <agent1>/	# Tasks claimed by agent 1
└─ <agent2>/	# Tasks claimed by agent 2
└─ Plans/	# Generated plans with analysis
└─ Done/	# Completed tasks
└─ Pending_Approval/	# Awaiting human approval
└─ Approved/	# User approved, ready to execute
└─ Rejected/	# User rejected actions
└─ Logs/	# Audit trail (daily JSON logs)
└─ Accounting/	# Financial records
└─ Current_Month.md	# Transaction ledger
└─ Invoices/	# Generated invoices
└─ Receipts/	# Payment receipts
└─ Dashboard.md	# Real-time status page
└─ Company_Handbook.md	# Rules and policies
└─ Business_Goals.md	# Quarterly objectives
└─ .env	# Environment variables (NEVER commit!)
└─ .gitignore	# Exclude secrets, creds
└─ requirements.txt	# Python dependencies
└─ setup.sh	# Setup script
└─ config.yaml	# Configuration
└─ README.md	# Project documentation

Setup Instructions

Step 1: Prerequisites

Copy

```
# Install Python 3.13+  
python3 --version # Should be 3.13 or higher
```

```
# Install Node.js 24+ LTS  
node --version # Should be 24+
```

```
# Install Obsidian  
# Download from https://obsidian.md/
```

```
# Install Claude Code  
npm install -g @anthropic/claude-code
```

```
# Verify installation  
claude --version
```

Step 2: Clone and Configure

Copy

```
# Clone this repository  
git clone <your-repo-url>  
cd ai_employee_project  
  
# Create Python virtual environment  
python3 -m venv venv  
source venv/bin/activate # On Windows: venv\Scripts\activate
```

```
# Install Python dependencies  
pip install -r requirements.txt
```

Step 3: Create Environment File

Copy

```
# Create .env file with your credentials  
cat > .env << 'EOF'  
# Google Gmail API  
GMAIL_CREDENTIALS=/path/to/google-credentials.json  
GMAIL_CLIENT_ID=your_client_id  
GMAIL_CLIENT_SECRET=your_secret
```

```
# Vault configuration
VAULT_PATH=/path/to/AI_Employee_Vault

# WhatsApp Session
WHATSAPP_SESSION=/path/to/whatsapp_session

# API Keys
OPENAI_API_KEY=sk-your-key
ANTHROPIC_API_KEY=your-key

# File Drop Folder
DROP_FOLDER=/path/to/drop_folder
EOF

# Add to .gitignore
echo ".env" >> .gitignore
```

Step 4: Create Obsidian Vault

Copy

```
# Create vault structure
mkdir -p
AI_Employee_Vault/{Needs_Action,Plans,Done,In_Progress,Pending_Approval,Approved,Rejected,
Logs,Accounting}

# Initialize Obsidian vault (open in Obsidian app)
# 1. Open Obsidian
# 2. "Create new vault" → select AI_Employee_Vault folder
# 3. Confirm
```

Step 5: Set Up Claude Code Skills

Copy

```
# Option A: Register GitHub repository as plugin marketplace
claude --skill-marketplace add anthropics/skills

# Option B: Copy skills locally
cp -r .claude/skills ~/.claude/skills/personal-ai-employee/

# Verify skills are discoverable
```

```
claude --skills list | grep vault-operations
```

Step 6: Start Watchers

Copy

```
# Terminal 1: Gmail Watcher
```

```
source venv/bin/activate
```

```
python watchers/gmail_watcher.py
```

```
# Terminal 2: WhatsApp Watcher
```

```
source venv/bin/activate
```

```
python watchers/whatsapp_watcher.py
```

```
# Terminal 3: File System Watcher
```

```
source venv/bin/activate
```

```
python watchers/filesystem_watcher.py
```

```
# Terminal 4: Orchestrator
```

```
source venv/bin/activate
```

```
python orchestrator.py
```

Step 7: Test Skills Manually

Copy

```
# Open Claude Code in your vault directory
```

```
cd AI_Employee_Vault
```

```
claude
```

```
# In Claude Code, test skills:
```

```
/vault-operations
```

```
Process all items in /Needs_Action/ and create plans
```

```
/financial-auditor
```

```
Generate a CEO briefing for this week
```

```
# Verify files created:
```

```
ls -la Needs_Action/
```

```
ls -la Plans/
```

```
ls -la Approved/
```

Dependencies

File: requirements.txt

```
# Core
python-dotenv==1.0.0
pydantic==2.5.0

# Gmail
google-auth-oauthlib==1.2.0
google-auth-httpplib2==0.2.0
google-api-python-client==2.100.0

# WhatsApp
playwright==1.40.0

# Scheduling
schedule==1.2.0

# File watching
watchdog==3.0.0

# Utilities
pytz==2023.3
requests==2.31.0
```

Tier-Based Delivery Roadmap

Bronze Tier: Foundation (8-12 hours)

Goal: Prove the architecture works with minimal functionality.

Deliverables:

-  Create `.claude/skills/vault-operations/SKILL.md`
 - Read from vault

- Create plans
 - Update dashboard
-  Implement watchers/base_watcher.py + Gmail Watcher
 - Monitor Gmail
 - Create EMAIL_*.md files
-  Create Obsidian vault structure
 - /Needs_Action/
 - /Plans/
 - /Done/
 - /Dashboard.md
 - /Company_Handbook.md
-  Basic Orchestrator
 - Call Claude with skills
 - Schedule tasks
-  Audit logging
 - Log to /Logs/YYYY-MM-DD.json

Bronze Tier Demo: "Email arrives → Gmail Processor skill triages it → creates Plan → moves to Done"

Success Criteria:

- Can read email from Gmail
- Can create action files
- Claude can process them with skills
- Files move through state machine

Silver Tier: Functional Assistant (20-30 hours)

Goal: Create a useful, working Personal AI Employee.

Deliverables:

-  All Bronze requirements
-  Implement remaining core skills:
 - Gmail Processor (categorize, draft responses)
 - WhatsApp Handler (handle urgent messages)
 - Financial Auditor (skeleton, weekly revenue only)
 - Approval Workflow (basic HITL)
-  Add WhatsApp Watcher
 - Playwright monitoring
 - Detect urgent keywords
 - Create WHATSAPP_*.md files
-  Human-in-the-loop approval workflow
 - Create /Pending_Approval/ files
 - User moves to /Approved/ or /Rejected/
 - Orchestrator monitors and executes
-  Email MCP Server (basic)
 - Send emails to approved contacts
 - Execute from /Approved/ files
-  Scheduling
 - 2-hour inbox processing
 - 30-minute approval checks

Silver Tier Demo: "Email from known client requests invoice → Gmail Processor drafts response → creates approval request → user approves → Email MCP sends → logged"

Success Criteria:

- Multiple skills working together
 - WhatsApp monitoring active
 - Email sending works with approvals
 - HITL workflow prevents mistakes
-

Gold Tier: Autonomous Employee (40+ hours)

Goal: Achieve production-grade autonomy with comprehensive features.

Deliverables:

-  All Silver requirements
-  Complete Financial Auditor skill
 - Read transactions from Accounting
 - Calculate weekly/monthly revenue
 - Identify bottlenecks
 - Find cost savings
 - Generate full CEO Briefing
-  Ralph Wiggum Loop
 - Task completion detection
 - Multi-iteration persistence
 - Max iteration protection
-  Odoo Community Integration

- Self-hosted Odoo on local VM
 - Odoo MCP Server (read/write)
 - Invoice generation
 - Transaction syncing
-  Social Media Integration
 - Facebook skill + MCP
 - Instagram skill + MCP
 - Twitter skill + MCP
 - Post scheduling and drafting
-  Comprehensive error handling
 - Graceful degradation
 - Auto-recovery
 - Error quarantine
 - Human alerts
-  Advanced audit logging
 - 90-day retention
 - Structured JSON format
 - Searchable/queryable

Gold Tier Demo: "Sunday night → Monday 7am CEO Briefing generated automatically with revenue, completed tasks, bottlenecks, cost savings, and action items"

Success Criteria:

- Autonomous multi-step tasks
- Financial analysis working
- Social media posting
- Zero manual intervention for routine tasks

- Complete audit trail
-

Platinum Tier: Cloud + Local (60+ hours)

Goal: Production deployment with 24/7 operation and cloud/local coordination.

Deliverables:

-  All Gold requirements
-  Cloud VM Deployment
 - Oracle Cloud free tier (or AWS)
 - Cloud agent runs 24/7
 - Health monitoring and auto-restart
-  Work-Zone Specialization
 - **Cloud responsibilities:**
 - Email triage and drafting
 - Social media drafting
 - Invoice drafting (Odoo)
 - **Local responsibilities:**
 - Final approvals
 - WhatsApp session (never in cloud)
 - Banking credentials (never in cloud)
 - Final sends
-  Vault Synchronization
 - Git-based sync between Cloud and Local
 - Claim-by-move pattern (prevents double-work)
 - File-based handoff system

- Never sync secrets
-  Security Architecture
 - Secrets never leave local machine
 - Cloud never sees WhatsApp sessions
 - Cloud never sees banking credentials
 - Only markdown/state syncs
-  Odoo on Cloud
 - Deployed on Cloud VM with HTTPS
 - Auto-backups
 - Health monitoring
 - Cloud MCP integration for draft-only

Platinum Tier Demo: "Email arrives at 3am while local is offline → Cloud drafts reply in /Plans/ → User wakes up and approves → Local executes send and logs → system proves end-to-end async coordination"

Success Criteria:

- Cloud/local separation of concerns
- Zero credential leaks
- Always-on operation
- Reliable vault sync
- Production-grade reliability

Security & Best Practices

Credential Management

Never store credentials in plain text or in your Obsidian vault.

Use Environment Variables

Copy

```
# .env file (add to .gitignore immediately!)
GMAIL_CLIENT_ID=your_client_id
GMAIL_CLIENT_SECRET=your_secret
BANK_API_TOKEN=your_token
WHATSAPP_SESSION_PATH=/secure/path/session
```

Use System Credential Managers

Copy

```
# macOS Keychain
import keychain

token = keychain.get_password("service_name", "account_name")
keychain.set_password("service_name", "account_name", token)

# Windows Credential Manager
from win32cred import CredentialManager
cred = CredentialManager.read("vault://my-credential")

# Linux (pass)
import subprocess
password = subprocess.check_output(['pass', 'my-password']).decode().strip()
```

Rotate Credentials Regularly

Copy

```
# Schedule monthly rotation
schedule.every().month.do(rotate_all_credentials)

def rotate_all_credentials():
    # Regenerate Gmail OAuth tokens
    # Regenerate banking API tokens
```

```
# Regenerate WhatsApp sessions
logger.info("Credentials rotated")
```

Sandboxing & Isolation

Protect against unintended actions during development.

Development Mode Flag

Copy

```
# In every action script
DRY_RUN = os.getenv('DRY_RUN', 'true').lower() == 'true'

def send_email(to, subject, body):
    if DRY_RUN:
        logger.info(f'[DRY RUN] Would send email to {to}')
        return # Don't actually send

    # Real send logic here
    service.users().messages().send(...)
```

Use Test Accounts

Copy

```
# During development
export GMAIL_ACCOUNT=test+ai@example.com
export BANK_ACCOUNT=test_account_123
export WHATSAPP_SESSION=test_session

# In production
export GMAIL_ACCOUNT=business@example.com
export BANK_ACCOUNT=real_account_456
export WHATSAPP_SESSION=real_session
```

Rate Limiting

Copy

```

def send_email(to, subject, body):
    # Max 10 emails per hour
    emails_this_hour = count_sent_emails(datetime.now() - timedelta(hours=1))
    if emails_this_hour > 10:
        raise RateLimitError("Max 10 emails per hour")

    # Send email
    service.users().messages().send(...)

def make_payment(recipient, amount):
    # Max 3 payments per hour
    payments_this_hour = count_made_payments(datetime.now() - timedelta(hours=1))
    if payments_this_hour > 3:
        raise RateLimitError("Max 3 payments per hour")

    # Make payment
    api.make_payment(recipient, amount)

```

Audit Logging

Every action must be logged for review.

Log Format (JSON)

Copy

```

{
  "timestamp": "2026-01-28T10:30:00Z",
  "action_type": "email_send",
  "actor": "claude_code",
  "target": "client@example.com",
  "parameters": {
    "subject": "Invoice #123",
    "body_preview": "Please find attached..."
  },
  "approval_status": "approved",
  "approved_by": "human",
  "result": "success",
  "mcp_server": "email-mcp",
  "execution_time_ms": 245
}

```

Storage & Retention

Copy

```
def log_action(self, action_type, details, status, actor='claude'):
    log_file = self.logs / f'{datetime.now().strftime('%Y-%m-%d')}.json'

    entry = {
        'timestamp': datetime.now().isoformat(),
        'action_type': action_type,
        'actor': actor,
        'details': details,
        'status': status
    }

    # Append to daily log file
    entries = []
    if log_file.exists():
        entries = json.loads(log_file.read_text())

    entries.append(entry)
    log_file.write_text(json.dumps(entries, indent=2))

    # Retention: Keep 90 days
    self.cleanup_old_logs()

def cleanup_old_logs(self):
    cutoff = datetime.now() - timedelta(days=90)
    for log_file in self.logs.glob('*.json'):
        if log_file.stat().st_mtime < cutoff.timestamp():
            log_file.unlink()
```

Permission Boundaries

Not all actions require approval — but the right ones do.

Action Category	Auto-Approve Threshold	Always Require Approval
-----------------	------------------------	-------------------------

Email replies	To known contacts	New contacts, bulk sends
Payments	< \$50 recurring	All new payees, > \$100
Social media	Scheduled posts	Replies, DMs
File operations	Create, read	Delete, move outside vault

Error Recovery

Autonomous systems will fail. Plan for it.

Error Categories

Category	Examples	Recovery Strategy
Transient	Network timeout, API rate limit	Exponential backoff retry
Authentication	Expired token, revoked access	Alert human, pause operations

Logic	Claude misinterprets message	Human review queue
Data	Corrupted file, missing field	Quarantine + alert
System	Orchestrator crash, disk full	Watchdog + auto-restart

Retry Logic

Copy

```
from functools import wraps
import time
```

```
def with_retry(max_attempts=3, base_delay=1, max_delay=60):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(max_attempts):
                try:
                    return func(*args, **kwargs)
                except TransientError as e:
                    if attempt == max_attempts - 1:
                        raise
                    delay = min(base_delay * (2 ** attempt), max_delay)
                    logger.warning(f'Attempt {attempt+1} failed, retrying in {delay}s')
                    time.sleep(delay)
            return wrapper
        return decorator
```

```
@with_retry(max_attempts=3, base_delay=1)
def send_email_with_retry(to, subject, body):
    service.users().messages().send(...)
```

Graceful Degradation

Copy

```
def process_inbox():
    try:
        # Try Gmail
        gmail_items = fetch_gmail()
    except Exception as e:
        logger.warning(f"Gmail API down: {e}")
        gmail_items = [] # Continue without Gmail

    try:
        # Try WhatsApp
        whatsapp_items = fetch_whatsapp()
    except Exception as e:
        logger.warning(f"WhatsApp down: {e}")
        whatsapp_items = [] # Continue without WhatsApp

    # Process whatever we could get
    process_all_items(gmail_items + whatsapp_items)
```

Quick Start Guide

5-Minute Setup

Copy

```
# 1. Install prerequisites
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt

# 2. Create .env file
cp .env.example .env
# Edit .env with your API keys

# 3. Create vault
mkdir -p AI_Employee_Vault/{Needs_Action,Plans,Done,Logs}

# 4. Test basic skill
cd AI_Employee_Vault
claude
# Type: /vault-operations
```

5. Start watchers

```
python ../watchers/gmail_watcher.py &  
python ../watchers/whatsapp_watcher.py &
```

6. Start orchestrator

```
python ../orchestrator.py
```

First Task: Process Your Inbox

- **Create a test email** in Gmail (or use an existing one)
- **Mark as important** (so Gmail Watcher picks it up)
- **Wait 2-3 minutes** for Watcher to run
- **Check vault:** `ls AI_Employee_Vault/Needs_Action/`
- **See EMAIL_*.md file** created
- **Run Claude:** `claude /gmail-processor`
- **Check Plans:** `ls AI_Employee_Vault/Plans/`
- **See PLAN_*.md file** created

Result: Email successfully triaged and planned!

Scheduling First Briefing

- **Edit** `/Company_Handbook.md`:

Copy

Business Goals

- Monthly revenue target: \$10,000
- Client response SLA: 24 hours

- **Edit** `/Business_Goals.md`:

Copy

Q1 2026 Objectives

- Monthly goal: \$10,000
- Active projects: 2

- **Wait until Monday 8:00 AM** (or manually trigger):

Copy

```
claude --skill financial-auditor \  
"Generate CEO briefing"
```

- **Check:** `ls AI_Employee_Vault/Briefings/`

Result: CEO Briefing generated automatically!

Troubleshooting & FAQ

Setup Issues

Q: Claude Code says "command not found"

A: Ensure Claude Code is installed globally:

Copy

```
npm install -g @anthropic/claude-code  
# Restart terminal  
claude --version
```

Q: Obsidian vault isn't being read by Claude

A: Check file permissions and path:

Copy

Run Claude from vault directory

```
cd AI_Employee_Vault
```

```
claude
```

Or specify path explicitly

```
claude --cwd /path/to/AI_Employee_Vault
```

Q: Gmail API returns 403 Forbidden

A: Your OAuth consent screen may need verification:

- Go to Google Cloud Console
- API & Services → Gmail API
- Enable the API
- Create OAuth credentials (Desktop app)
- Re-generate credentials JSON

Q: Python dependencies won't install

A: Upgrade pip and try again:

Copy

```
pip install --upgrade pip
```

```
pip install --upgrade -r requirements.txt
```

Runtime Issues

Q: Watcher scripts stop running overnight

A: Use a process manager like PM2:

Copy

```
npm install -g pm2
```

```
pm2 start watchers/gmail_watcher.py --interpreter python3
pm2 save
pm2 startup
```

Q: Claude is making incorrect decisions

A: Review and improve your `Company_Handbook.md`:

- Add more specific rules
- Include examples
- Lower autonomy thresholds
- Require more approvals for uncertain actions

Q: MCP server won't connect

A: Debug the connection:

Copy

```
# Check server is running
ps aux | grep mcp
```

```
# Check logs
```

```
tail -f /path/to/logs/mcp.log
```

```
# Verify path in config
```

```
cat ~/.config/claude-code/mcp.json
```

Q: Tasks not moving to /Done/

A: Check Ralph Wiggum loop logic:

- Is Claude writing completion status?
- Is it moving the file?
- Check task file for errors

Copy

Debug

cat /Vault/TASK_*.md

cat /Vault/Done/TASK_*.md # Should have files

Security Concerns

Q: How do I know my credentials are safe?

A:

-  Never commit .env file
-  Use environment variables
-  Rotate credentials monthly
-  Implement audit logging (see logs)
-  Review Logs / directory weekly

Q: What if Claude tries to pay the wrong person?

A: That's why HITL is critical:

- All payments > \$100 require approval
- All new recipients require approval
- User must move file to /Approved/
- Claude cannot execute without user permission

Conclusion

You now have a complete blueprint for building an autonomous Digital FTE using Claude Code skills, Obsidian, and Python watchers.

Key Architectural Principles:

- **Local-first:** Keep sensitive data on your machine
- **Skills-based:** Use modular, reusable Agent Skills
- **Human-in-the-loop:** Require approval for risky actions
- **Audit everything:** Log all actions for review
- **Graceful degradation:** Handle failures elegantly

Start with Bronze Tier, prove the concept works, then **gradually add sophistication** through Silver, Gold, and Platinum tiers.

Remember: You're not just building software — you're hiring an AI employee. Treat it with the care and oversight you'd give to any team member.

Good luck, and see you at the Wednesday research meetings! 🚀

Document Version: 1.0

Last Updated: January 28, 2026

Next Review: February 4, 2026